FPT UNIVERSITY

Final Report

AI Speech-To-Text

Subject: CDP301

No	Name	Student ID
1	Phạm Minh Tùng	SE204386
2	Trần Tuấn Lâm	SE194892
3	Vũ Minh Quân	SE204813
4	Đỗ Phước Hùng	SE204688
5	Nguyễn Nam Khánh	SE205204

May, 2026

TABLE OF CONTENTS

1. Introduction 5

1.1 Problem Statement 5

1.2 Objectives 5

2. System Block Diagram 7

2.1 Overall System Block Diagram 7

2.2 Data Flow 10

3 Component Selection 13

3.1 ESP32-WROOM-32UE 13

3.2 INMP441 Digital Microphone 13

3.3 AMS1117 Voltage Regulator 14

3.4 CH340C USB-to-UART 14

3.5 Status LEDs 15

3.6 SSD1306 OLED Display 15

3.7 W25Q256JVEIQ.....16

---

4. Literature Review 16

4.1 I2S Digital Audio Communication 16

4.2 TinyML and Edge Impulse 16

4.3 MFE Feature Extraction 17

4.4 CNN Architecture 17

5. Operating Principle 19

6. Schematic 20

7. PCB Layout 28

8. Dataset and Training 38

Part 1: PCB Hardware Development Pipeline……38
Part 2: TinyML Machine Learning Pipeline……...39

9. Firmware & Source Code 47

9.1 I2S Initialization 47

9.2 Audio Recording 48

9.3 CNN Inference and Prediction 49

9.4 OLED Display 49

9.5 Main Program 50

10. Project Results 51

10.1 Standby State 51

10.2 Audio Recording and Processing 52

10.3 Recognition Result 53

10.4 Experimental Summary 54

11. Conclusion 54

1. Introduction

1.1 Problem Statement

Current Challenges: In academic and professional environments, the demand for transcribing spoken audio into text—such as meeting minutes, lecture notes, and video subtitles—is growing rapidly. However, manual transcription is incredibly time-consuming, labor-intensive, and prone to human error. Meanwhile, standard recording tools only store raw audio data, making it impossible to quickly search, edit, or digitize the spoken content.

The Solution & Project Rationale: To address these inefficiencies, developing a dedicated **Speech-to-Text AI Tool** is essential. By leveraging advanced natural language processing, this tool automates the transcription process, converting speech into accurate, context-aware text within seconds. This solution effectively eliminates manual bottlenecks, drastically boosts productivity, and seamlessly digitizes audio resources for better data management.

1.2 Objective

Because of the aforementioned issues, we decided to design and develop an embedded TinyML speech recognition system using the ESP32 chip and combining it with the INMP441 module, capable of performing offline speech keyword detection and displaying recognition results in real time via an OLED screen.

- Specific Objectives:
    1. Design and build an embedded speech recognition system using the Espressif Systems ESP32-32UE-WROOM-32UE microcontroller combined with the INMP441 digital microphone.
    2. Design a complete PCB circuit including the power supply, USB-UART interface, I2S microphone block, and system control block.
    3. Develop an embedded program for the ESP32-32UE to perform audio signal acquisition via I2S communication.
    4. Train the TinyML model using the Edge Impulse platform to recognize basic speech keywords.
    5. Deploy the TinyML model to the ESP32-S3 to perform offline speech recognition without an internet connection.
    6. Display keyword recognition results via Serial Monitor or OLE display.

    7. Design a status LED system including:

       The red LED indicates that the user is recording speech into the system.

The yellow LED indicates the status of recording or processing audio data.

The green LED indicates that the system has output the corresponding audio recording as text to the screen.

8. The system's performance is evaluated based on recognition accuracy, response speed, and PCB hardware stability.

2. System Block Diagram

2.1 Overall System Block Diagram

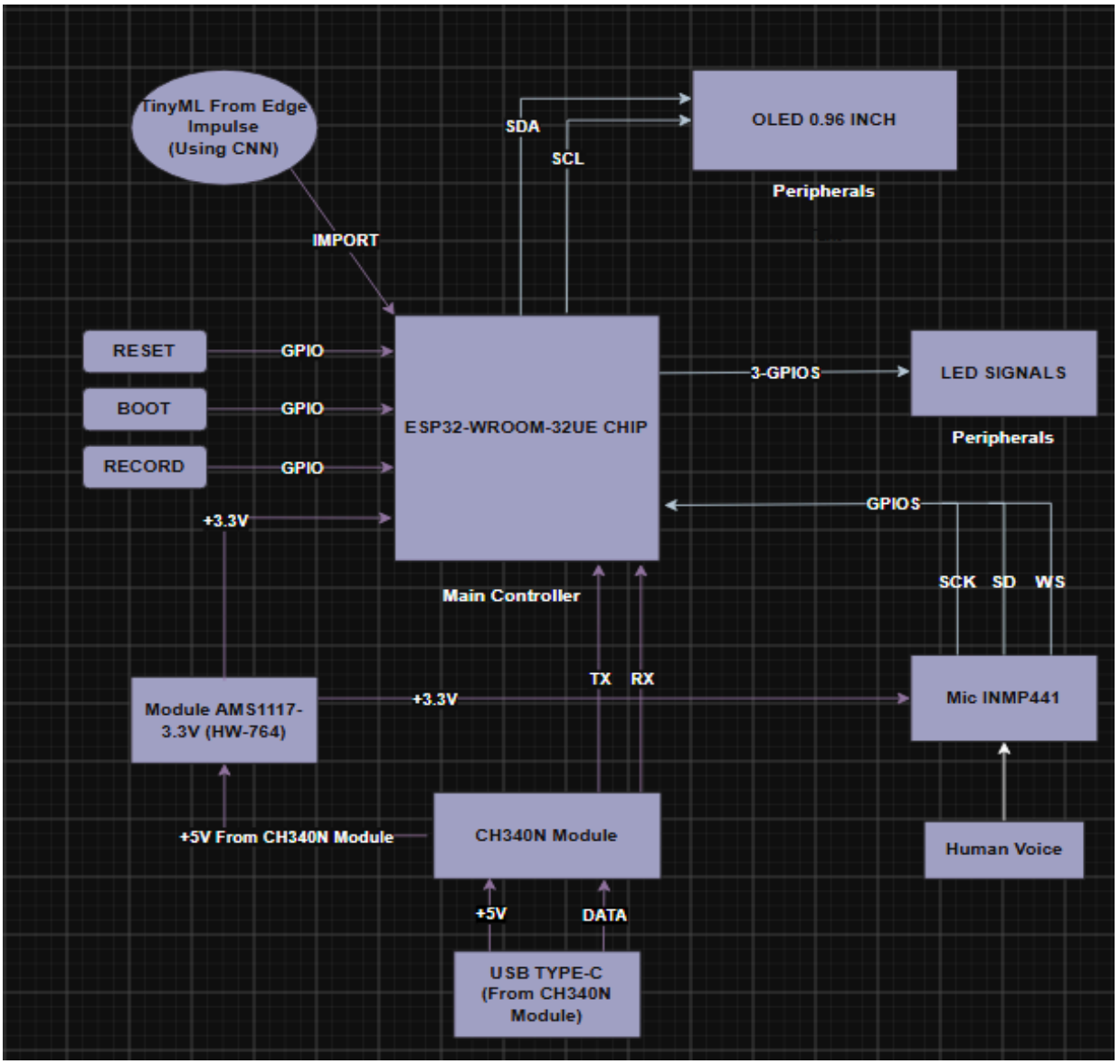


Figure 2.1. Overall System Block Diagram of the proposed TinyML-based speech recognition system.

ESP32-WROOM-32UE (Main Controller)

The ESP32-WROOM-32UE acts as the core processing unit of the system. It receives digital audio data from the INMP441 microphone through the I2S interface, performs MFE feature extraction, executes the TinyML CNN model, and controls all peripheral devices including the OLED display and status LEDs. In addition, it manages the user buttons (Record, Boot, and Reset) and communicates with the PC through the CH340N USB-to-UART interface for firmware programming and debugging.

INMP441 Digital Microphone

The INMP441 is responsible for capturing the user's speech and converting it into digital PCM audio samples. The microphone communicates with the ESP32 via the I2S protocol using the SCK, WS, and SD signals, providing high-quality digital audio while eliminating the need for an external analog-to-digital converter (ADC).

TinyML Model (CNN - Edge Impulse)

The speech recognition model is developed and trained using the Edge Impulse platform. After training, the optimized CNN model is exported as a C++ library and integrated into the ESP32 firmware. During operation, the ESP32 executes the imported model locally to recognize spoken keywords without requiring an internet connection.

SSD1306 OLED Display

The SSD1306 0.96-inch OLED serves as the primary output interface of the system. After successful inference, the recognized keyword is transmitted from the ESP32 to the OLED through the I2C interface (SDA and SCL), allowing the recognition result to be displayed in real time.

Status LEDs

Three status LEDs provide visual feedback regarding the operating state of the system.

- **Red LED:** Audio recording is in progress.
- **Yellow LED:** Audio preprocessing and AI inference are being performed.
- **Green LED:** Speech recognition has been completed successfully.

These indicators allow users to monitor the processing stages without requiring additional debugging tools.

CH340N USB-to-UART Module

The CH340N module provides the communication bridge between the ESP32 and a personal computer. It enables firmware uploading, serial debugging, and supplies the 5V USB power required by the system.

AMS1117 Voltage Regulator

The AMS1117 linear voltage regulator converts the incoming 5V USB supply into a stable 3.3V output required by the ESP32, INMP441 microphone, OLED display, and other low-voltage peripherals.

Control Buttons

The system includes three push buttons for user interaction.

- **Reset:** Restarts the ESP32.
- **Boot:** Places the ESP32 into firmware download mode.
- **Record:** Starts and stops voice recording for speech recognition.

2.2 Data Flow

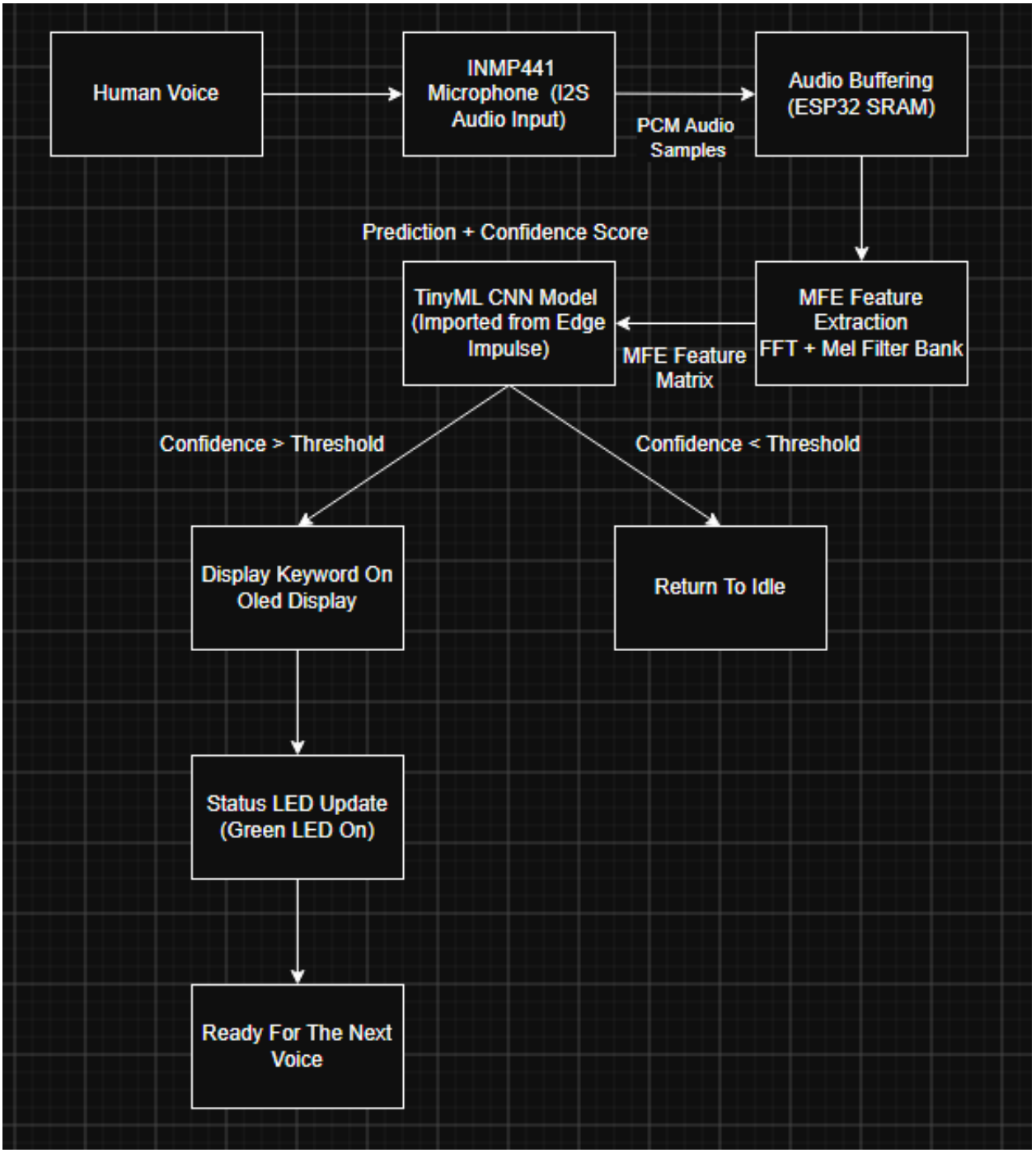


Figure 2.2 *presents the internal data processing flow of the proposed TinyML speech recognition system. The audio signal passes through several processing stages before the recognized keyword is displayed to the user.*

Step 1 – Voice Acquisition

The user speaks a predefined keyword into the INMP441 digital microphone. The microphone converts the acoustic signal into digital PCM audio samples and continuously transmits them to the ESP32 through the I2S interface.

Step 2 – Audio Buffering

The ESP32 receives the PCM audio stream and stores the samples in an internal memory buffer until the required recording duration has been collected.

Step 3 – Feature Extraction (MFE)

The buffered audio samples are processed using the Mel-Frequency Energy (MFE) algorithm. The preprocessing consists of framing the audio signal, applying the Fast Fourier Transform (FFT), passing the spectrum through a Mel filter bank, and calculating the energy of each Mel filter. The output of this stage is an MFE feature matrix that represents the speech signal in the time-frequency domain.

Step 4 – CNN Inference

The TinyML CNN model analyzes the extracted MFE feature matrix and produces a probability score for each predefined speech keyword. The keyword with the highest probability is selected as the predicted label together with its corresponding confidence score.

Step 5 – Decision Making

The ESP32 compares the confidence score with a predefined threshold. If the confidence exceeds the threshold, the prediction is accepted; otherwise, the input is classified as an unknown and noise.

Step 6 – User Feedback

The recognized keyword and its confidence score (in percentage) are displayed on the SSD1306 OLED display. At the same time, the corresponding status LED indicates the current operating state of the system. After displaying the result, the audio buffer is cleared and the system returns to the idle state.

3. Component Selection

3.1 **ESP32-WROOM-32UE Microcontroller:** Featuring a 240 MHz dual-core CPU and 520 KB SRAM, this chip provides the necessary processing power and memory to execute compacted machine learning models (TensorFlow Lite) locally (Edge AI) without internet dependency. The "UE" variant supports an external antenna, ensuring robust wireless communication stability when the device is enclosed.



3.2 **INMP441 Digital Microphone:** This module captures and transmits 24-bit digital audio directly via the I2S interface to the ESP32, completely eliminating the power-supply noise typical of traditional analog microphones. Delivering clean, high-quality audio signals is essential for the TinyML model to extract accurate acoustic features and achieve high recognition rates.



3.3 **AMS1117 3.3VDC Power Supply:** Capable of delivering a stable output current of up to 800mA, this LDO regulator keeps the system voltage smooth and flat. It prevents voltage drops that could distort microphone signals or cause microcontroller resets during spikes in power consumption when the ESP32 runs heavy AI inference or operates wireless radios.



3.4 **CH340C USB-to-UART Converter:** This IC establishes a reliable, high-speed serial connection between the microcontroller and the PC for seamless firmware flashing and debugging. It plays a critical role during the development phase, enabling the transmission of raw audio recorded directly from the hardware to the PC to build the training dataset.



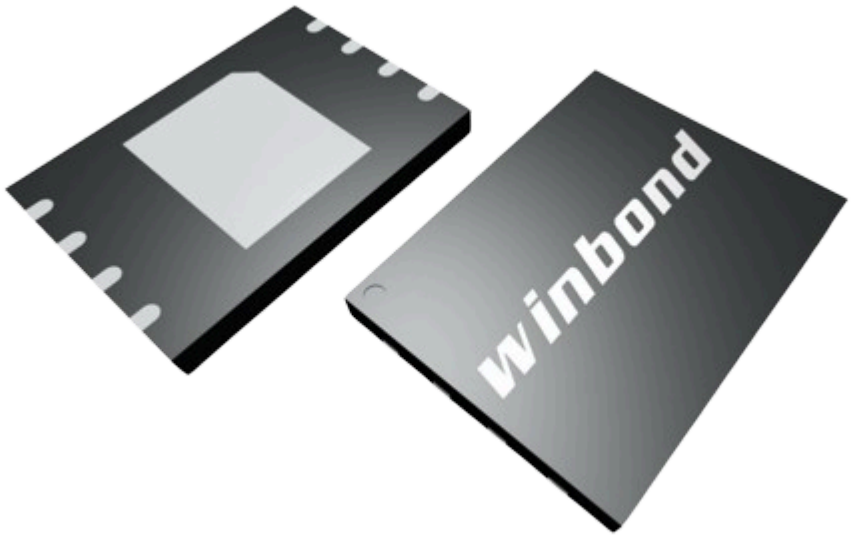
3.5 **Status LED Indicator:** The LED serves as a simple, low-overhead user interface to provide real-time visual feedback on the AI processing cycle. By toggling its state, it immediately informs the user whether the system is sampling audio, running inference, or successfully recognizing a command without wasting valuable computational resources.



3.6 **SSD1306 0.96-inch OLED Display:** This display establishes the system as a fully standalone Edge AI device by instantly rendering the transcribed text output upon successful recognition. Utilizing the 2-wire I2C protocol to save I/O pins, it offers sharp contrast and ultra-low power consumption, maximizing the overall power efficiency of the circuit.



3.7 **W25Q256JVEIQ:** is a **256 Mbit (32 MB) Serial NOR Flash memory** that communicates with the ESP32 through the SPI interface. It provides non-volatile storage for firmware, application data, and AI-related resources while offering high read performance and low power consumption.



4. Literature Review

4.1 I2S (Inter-IC Sound) Digital Audio Communication

Unlike conventional analog microphones, the INMP441 uses the I2S communication standard. This is a dedicated serial communication standard for transmitting digital audio data.

- SCK (Serial Clock): Bit synchronization clock.u
- WS (Word Select): Determines whether the data belongs to the left or right channel.
- SD (Serial Data): Audio data in PCM (Pulse Code Modulation) encoding.

Reason for use: Reduces environmental noise compared to analog signals and eliminates the need for a separate ADC, allowing the ESP32 to receive "clean" audio data for more accurate AI processing.

4.2 TinyML and Edge Impulse technologies

- TinyML: A field of Machine Learning focused on bringing ultra-small machine learning models to microcontrollers (MCUs) with limited RAM and Flash resources.
- Edge Impulse: A development platform that allows data collection, feature extraction, and model training to output C++ libraries compatible with ESP32.

4.3 MFE Feature Extraction

MFE (Mel-Frequency Energy) is a feature extraction technique that converts an audio signal into a time-frequency representation using the Mel frequency scale.

The extraction process consists of:

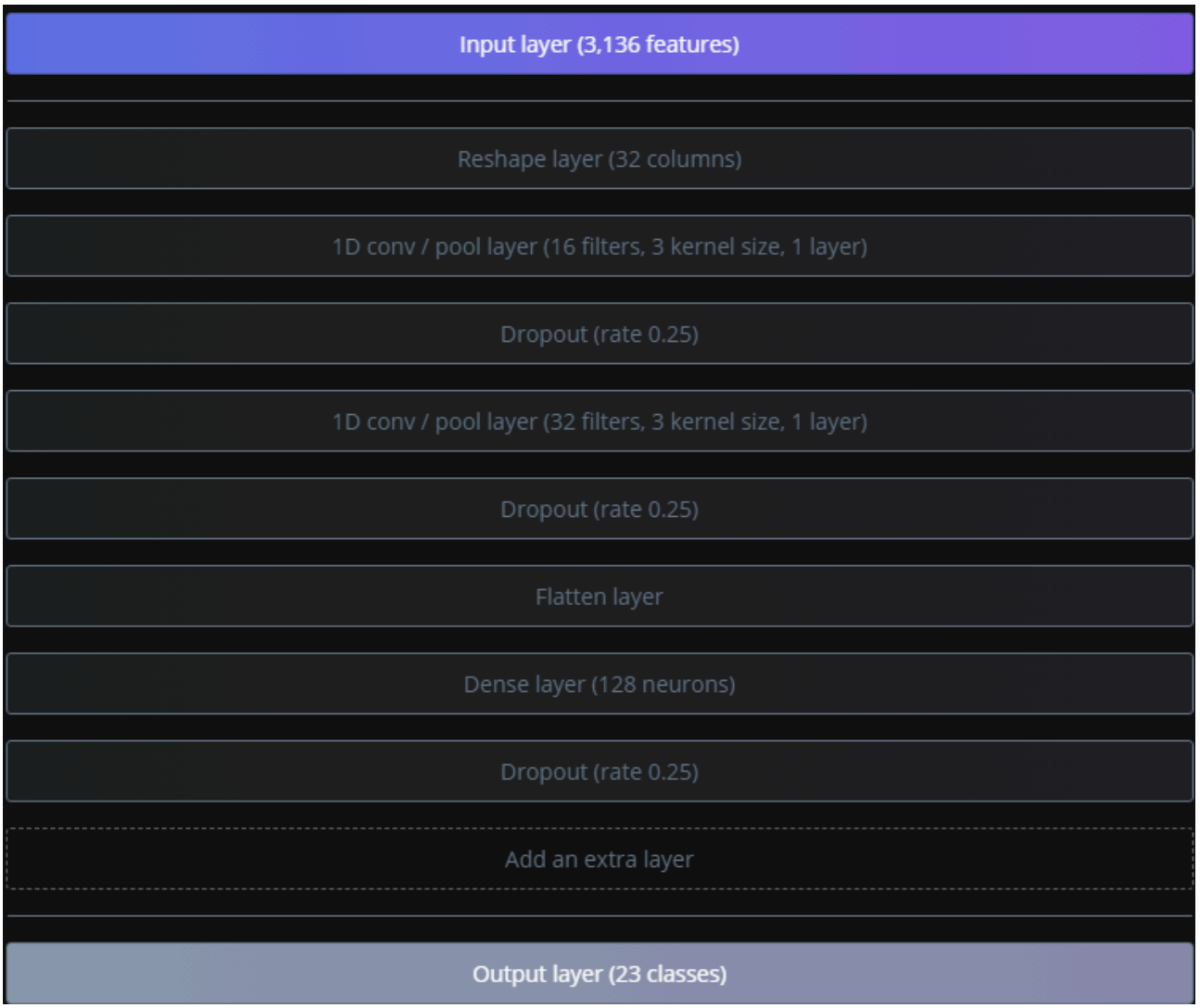
- Dividing the audio signal into short time frames.
- Converting each frame from the time domain to the frequency domain using the Fast Fourier Transform (FFT).
- Applying a Mel filter bank to model how the human ear perceives different frequencies.
- Calculating the energy in each Mel filter to generate a Mel-Frequency Energy (MFE) spectrogram.

Purpose: To transform the raw audio signal into a compact time-frequency representation while preserving the energy distribution across Mel frequency bands. This representation captures the important characteristics of speech and reduces the impact of noise before being used as the input to the neural network.

Reason for using MFE instead of MFCC: Unlike MFCC, which applies an additional Discrete Cosine Transform (DCT) to compress the Mel energies into cepstral coefficients, MFE retains the original Mel filter bank energies. This preserves more detailed spectral information, which is beneficial for convolutional neural networks (CNNs) that learn local spatial patterns from spectrogram-like inputs. In addition, MFE has lower computational complexity because it omits the DCT step, making it more suitable for TinyML and resource-constrained embedded systems such as the ESP32.

4.4 CNN Architecture

The proposed speech recognition system employs a lightweight one-dimensional Convolutional Neural Network (1D CNN) generated using the Edge Impulse platform. The network is specifically designed for TinyML deployment on the ESP32, providing a balance between recognition accuracy and computational efficiency.



As illustrated in **Figure 4.1**, the network consists of the following layers:

- **Reshape Layer:** Reshapes the extracted MFE feature matrix into the format required by the convolutional layers.
- **First 1D Convolution Layer:** Applies 16 convolution filters with a kernel size of 3 to extract low-level speech features.
- **Dropout Layer (0.25):** Randomly deactivates 25% of neurons during training to reduce overfitting.
- **Second 1D Convolution Layer:** Uses 32 filters with a kernel size of 3 to learn more complex speech patterns.
- **Dropout Layer (0.25):** Further improves the model's generalization capability.
- **Flatten Layer:** Converts the extracted feature maps into a one-dimensional feature vector.

- **Dense Layer:** Contains 128 neurons that perform high-level feature learning before classification.
- **Dropout Layer (0.25):** Prevents overfitting prior to the final classification layer automatically generated by Edge Impulse.

This lightweight architecture is suitable for resource-constrained embedded devices because it requires relatively low memory while maintaining sufficient recognition accuracy for keyword spotting applications.

5. Operating Principle

The operating principle of the proposed TinyML-based speech recognition system is illustrated through the following sequential steps.

Step 1: Voice Recording

When the user presses the **Record** button, the ESP32 initializes the I2S peripheral and starts receiving digital audio data from the INMP441 microphone. The incoming PCM audio samples are continuously stored in an internal buffer. During this stage, the **Red LED** is turned on to indicate that voice recording is in progress.

Step 2: Feature Extraction

After the recording is completed, the ESP32 preprocesses the buffered audio by applying the **Mel-Frequency Energy (MFE)** algorithm. The preprocessing includes framing the audio signal, performing the Fast Fourier Transform (FFT), applying the Mel filter bank, and computing the Mel-frequency energy matrix. During this processing stage, the **Yellow LED** is activated.

Step 3: TinyML Inference

The extracted MFE feature matrix is provided as the input to the embedded TinyML convolutional neural network (CNN). The model calculates the probability of each predefined speech keyword and selects the label with the highest confidence score. If the confidence value exceeds the predefined threshold, the prediction is accepted as the recognition result.

Step 4: Display and Feedback

The recognized keyword together with its confidence percentage is displayed on the SSD1306 OLED screen through the I2C interface. At the same time, the **Green LED** is turned on to indicate successful recognition. Finally, the audio buffer is cleared and the ESP32 returns to the idle state, ready to process the next speech command.

6. Schematic

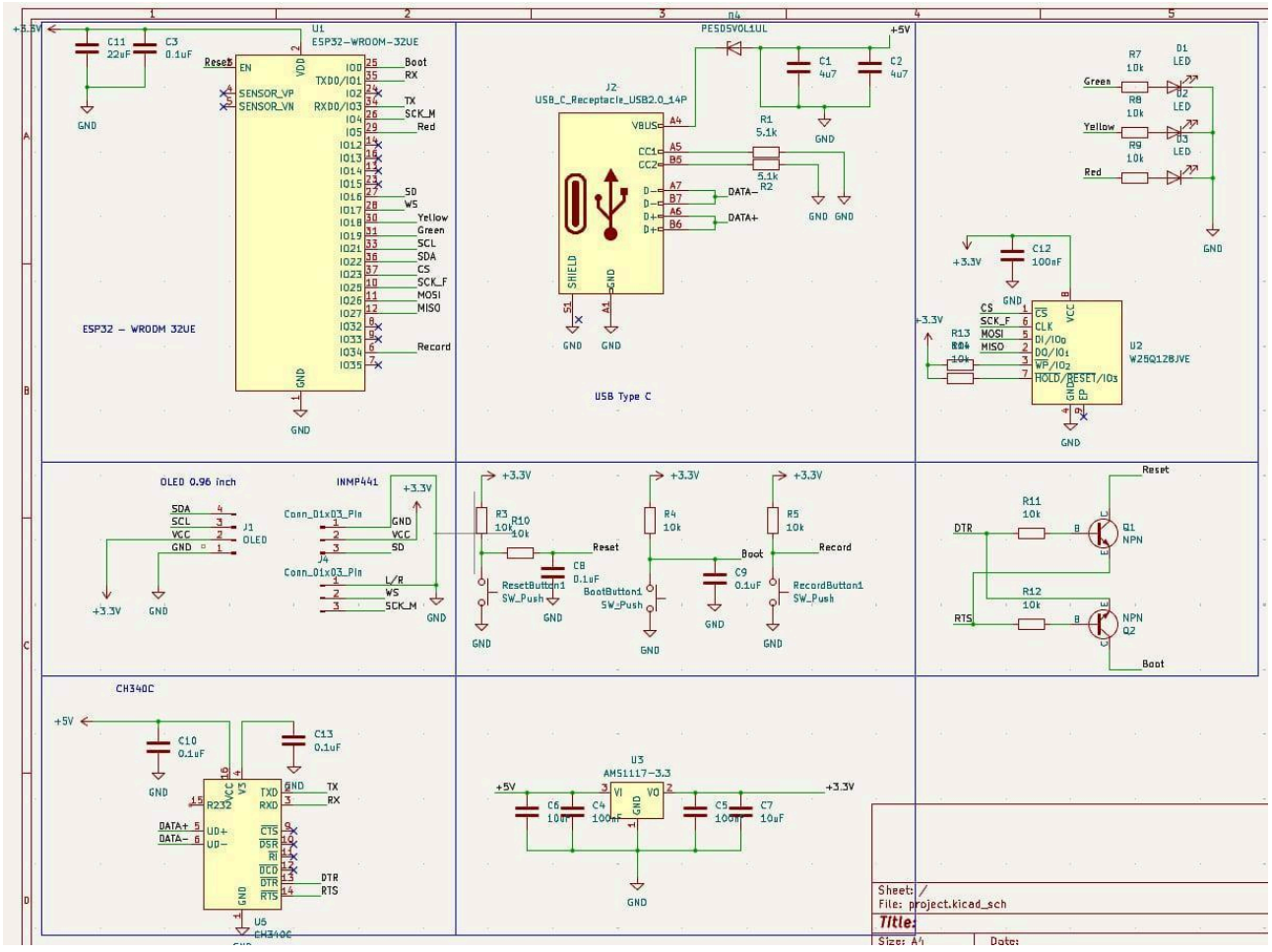


Figure 6.1

Description: Full schematic sheet showing all sub-circuits of the board — ESP32 core, USB-C input, status LEDs, SPI flash, OLED/microphone headers, buttons, auto-reset circuit, CH340C bridge, and the AMS1117 power supply — wired together on one page.

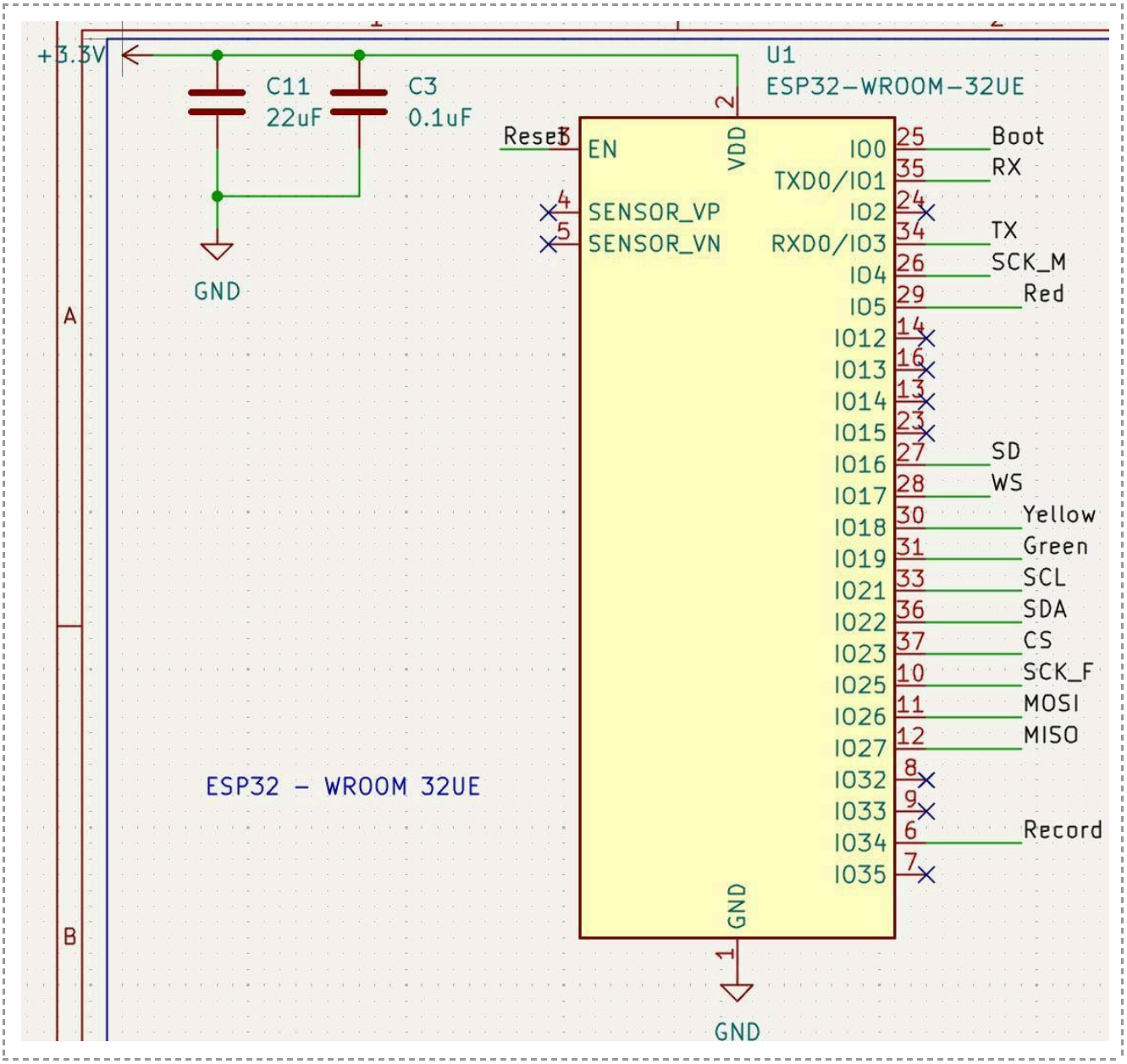


Figure 6.2

Description: Main ESP32-WROOM-32UE block (U1) with its power decoupling (C11 22µF, C3 0.1µF) and EN/Reset pin, showing every GPIO net assignment used on this board (Boot, RX, TX, SCK_M, Red, SD, WS, Yellow, Green, SCL, SDA, CS, SCK_F, MOSI, MISO, Record).

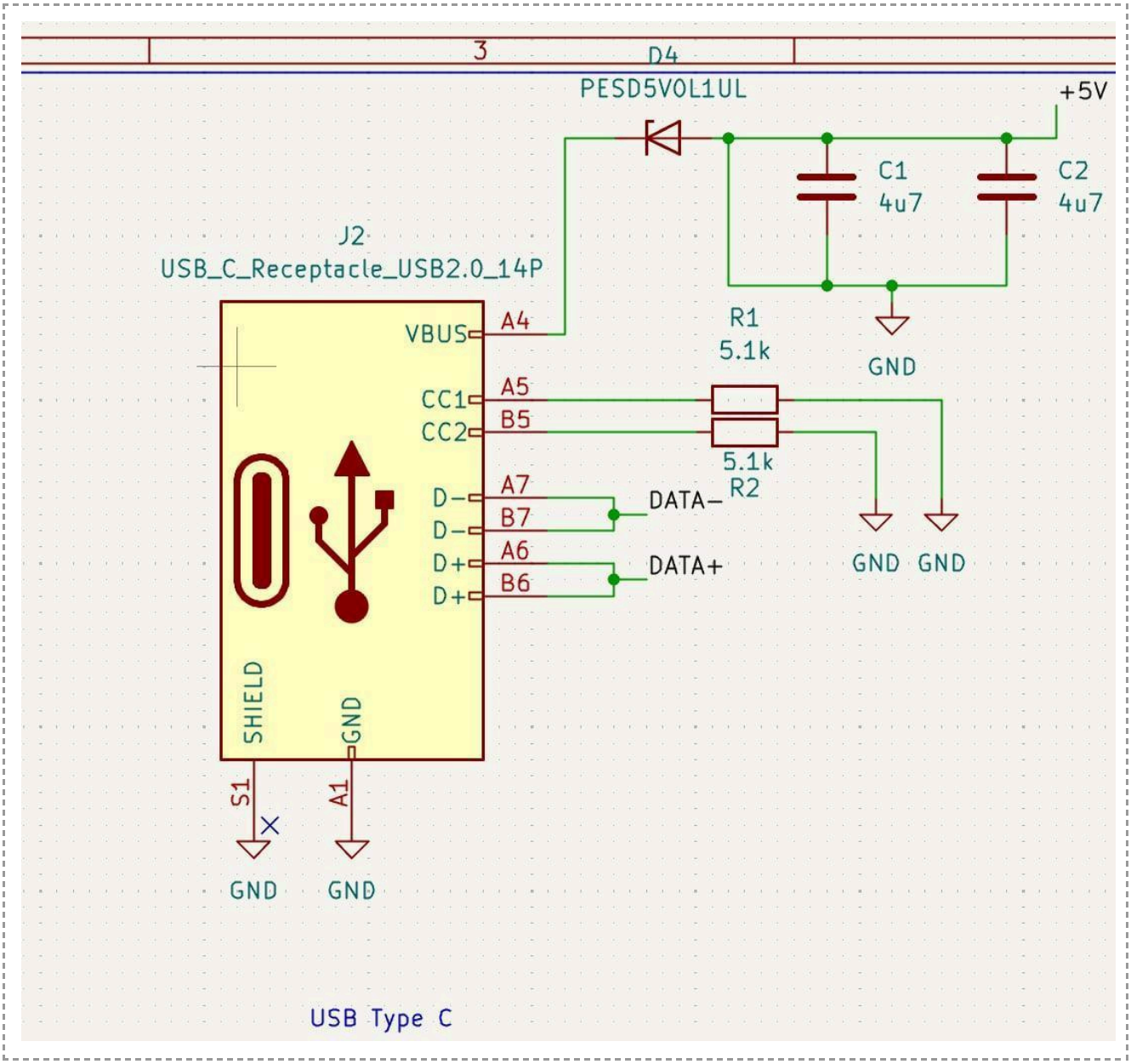


Figure 6.3

Description: USB Type-C receptacle (J2) circuit: 5V input protected by an ESD diode (D4) and filter caps (C1, C2 4.7µF), with CC1/CC2 configuration resistors (R1, R2 5.1k) and the USB D+/D- data lines routed to the CH340C.

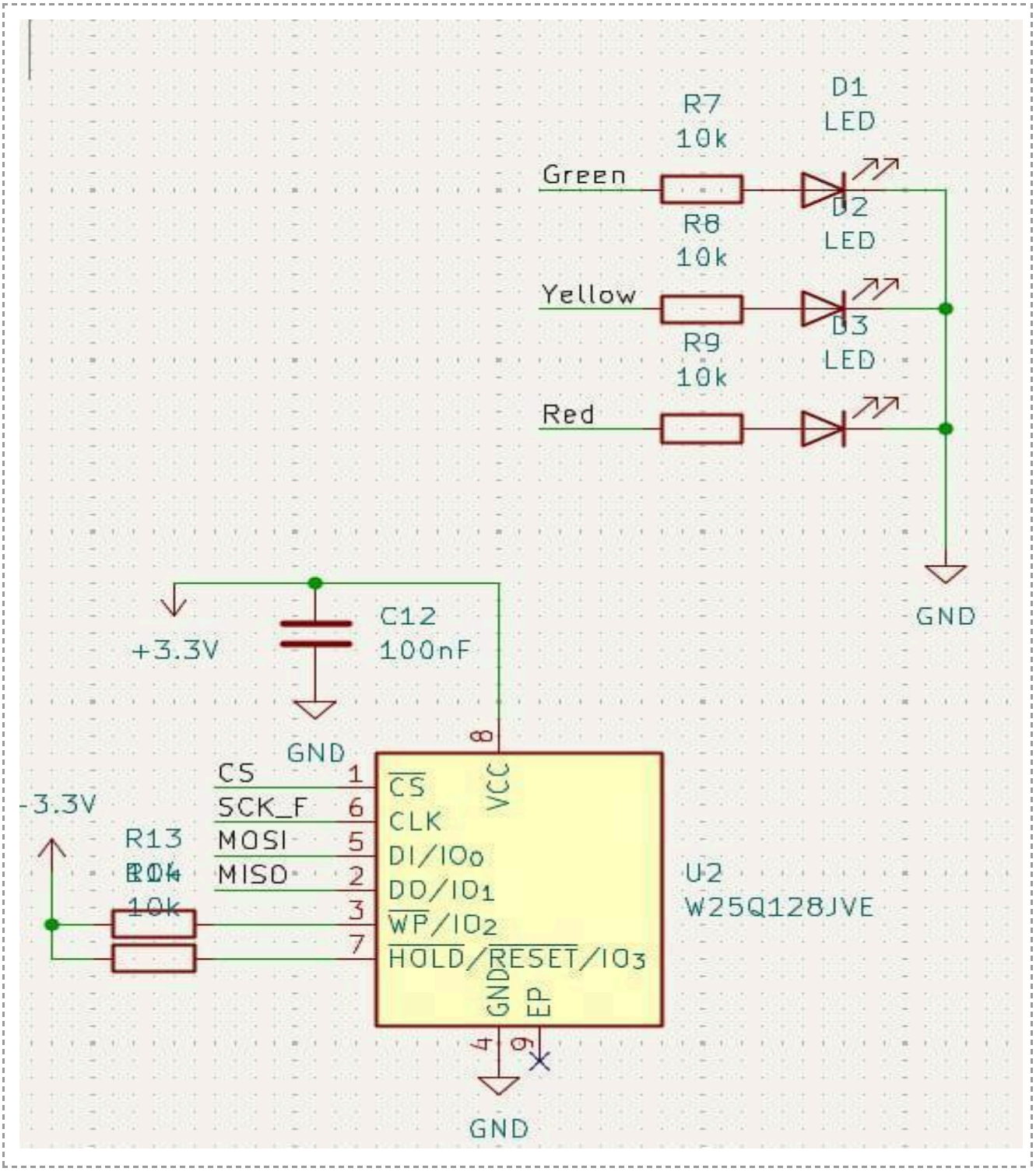


Figure 6.4

Description: Status LED driver network (D1 Green, D2 Red, D3 Yellow through current-limiting resistors R7–R9) alongside the W25Q128JVE SPI flash memory (U2) with its CS pull-up (R13/R14) for storing the compiled TinyML model.

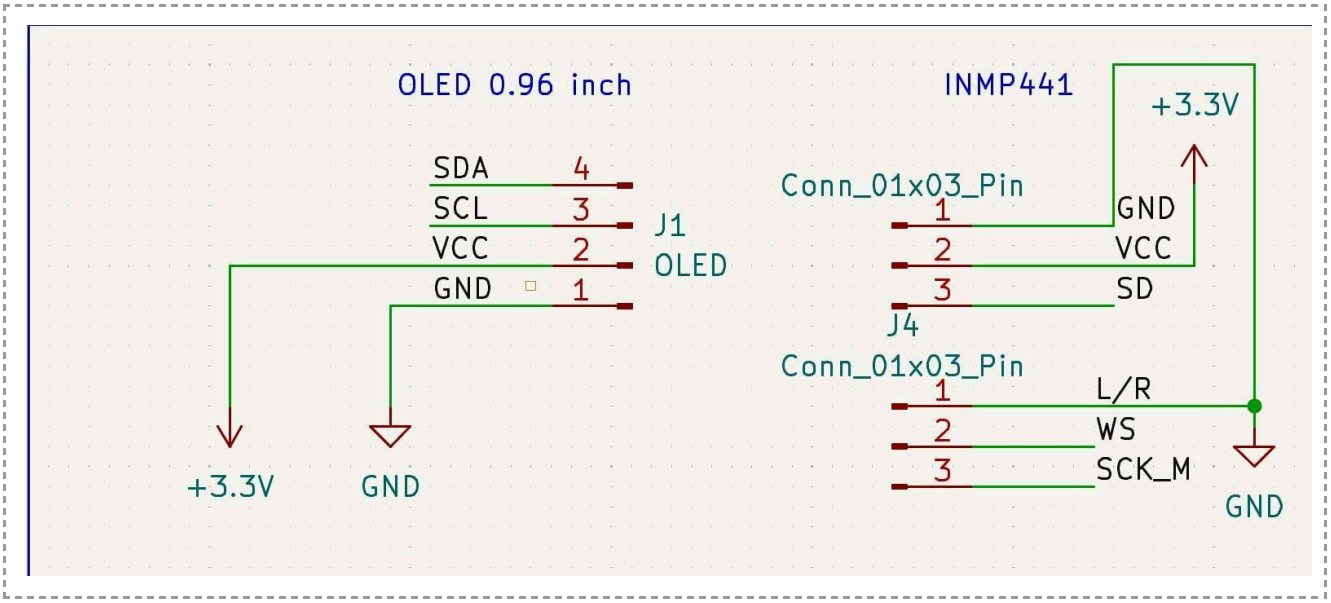


Figure 6.5

Description: Peripheral headers for the SSD1306 OLED (J1: SDA, SCL, VCC, GND) and the INMP441 I2S microphone (J4: GND/VCC/SD and L/R, WS, SCK_M), both referenced to the 3.3V rail.

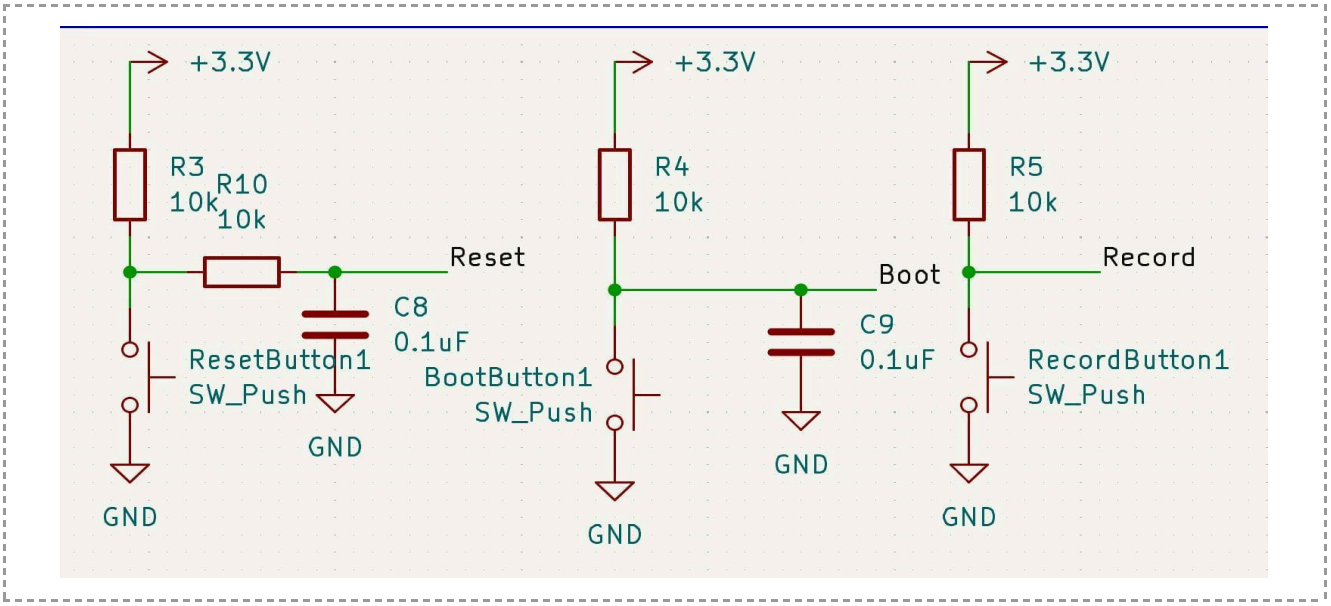


Figure 6.6

Description: Debounced push-button inputs for Reset, Boot, and Record, each using a 10k pull-up resistor (R3–R5) and a 0.1µF capacitor (C8, C9) to suppress mechanical bounce.

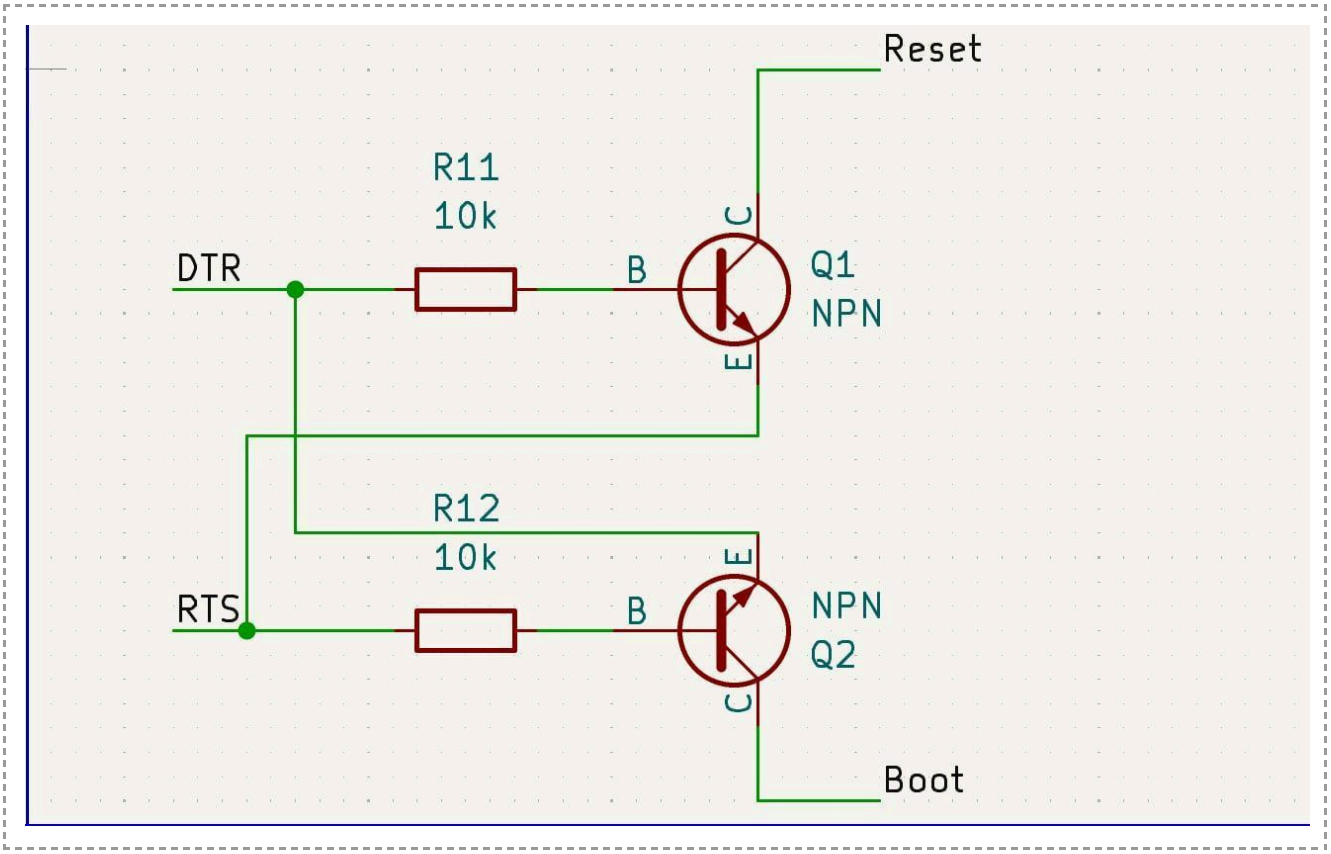


Figure 6.7

Description: Auto-program/auto-reset circuit: two NPN transistors (Q1, Q2) driven by the CH340C's DTR and RTS lines, generating the Reset and Boot control signals so the board can be flashed without manual button presses.

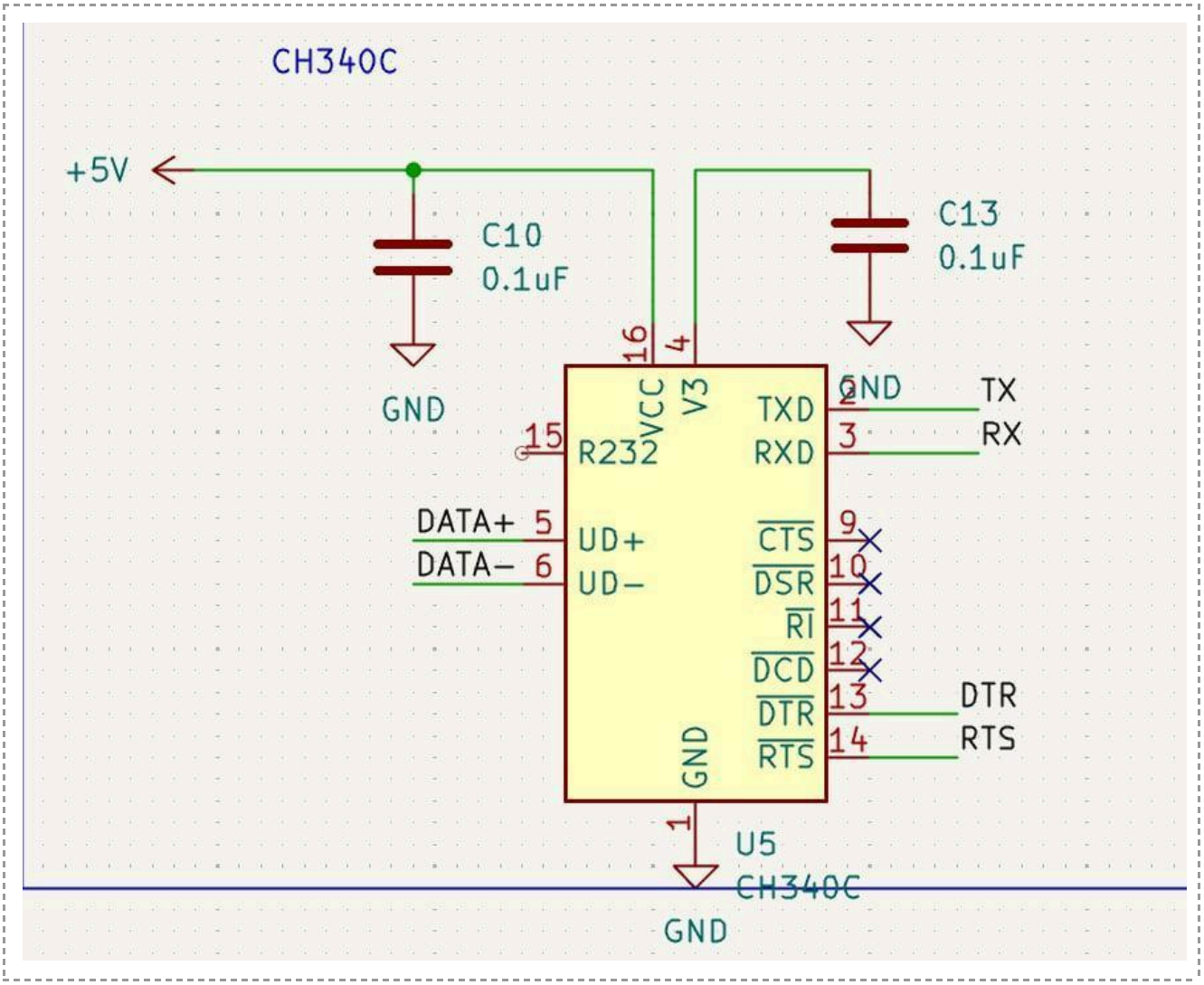


Figure 6.8

Description: CH340C USB-to-UART bridge (U5) with input/output decoupling (C10, C13 0.1µF), TXD/RXD lines to the ESP32, DTR/RTS handshake outputs, and the USB D+/D- differential pair.

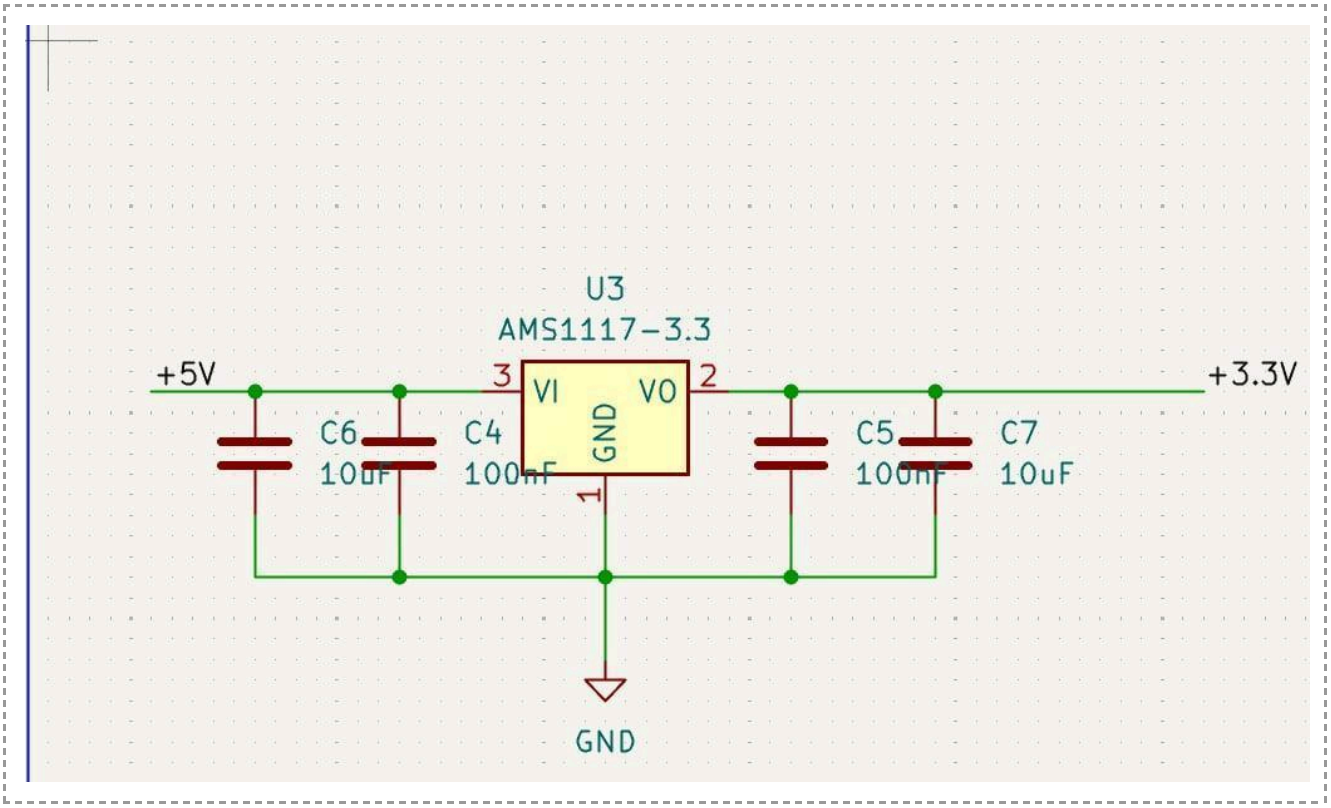


Figure 6.9

Description: AMS1117-3.3 linear regulator (U3) converting the 5V USB rail to a clean 3.3V supply, with input capacitors C4/C6 (100nF) and output capacitors C5/C7 (10µF) for ripple filtering.

7. PCB Layout

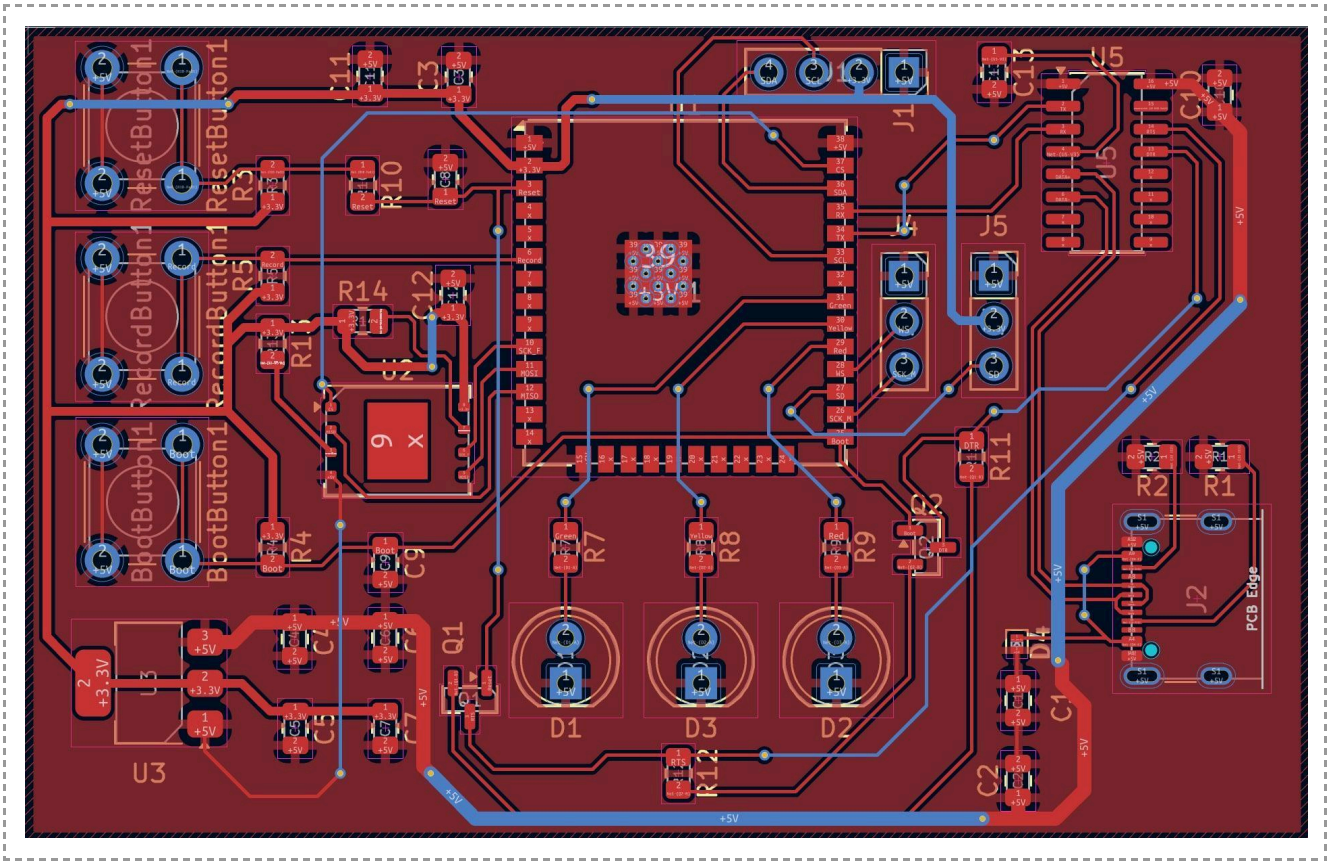


Figure 7.1

Description: Top copper layer of the assembled PCB layout, showing the placement of the ESP32 module, USB-C connector, buttons, status LEDs, and the SPI flash chip.

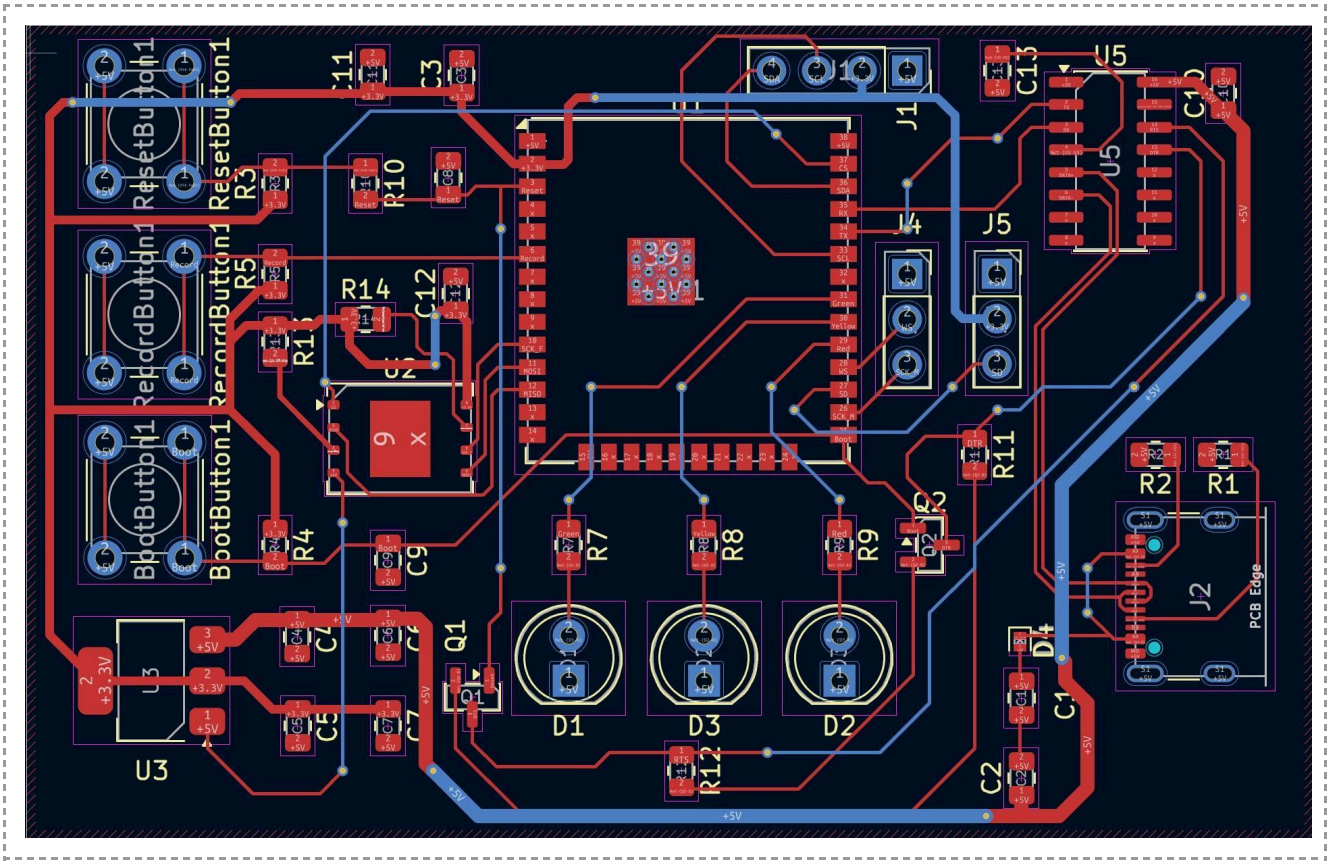


Figure 7.2

Description: Alternate render of the PCB layout highlighting trace routing between the main functional blocks (power, USB, buttons, LEDs, flash memory).

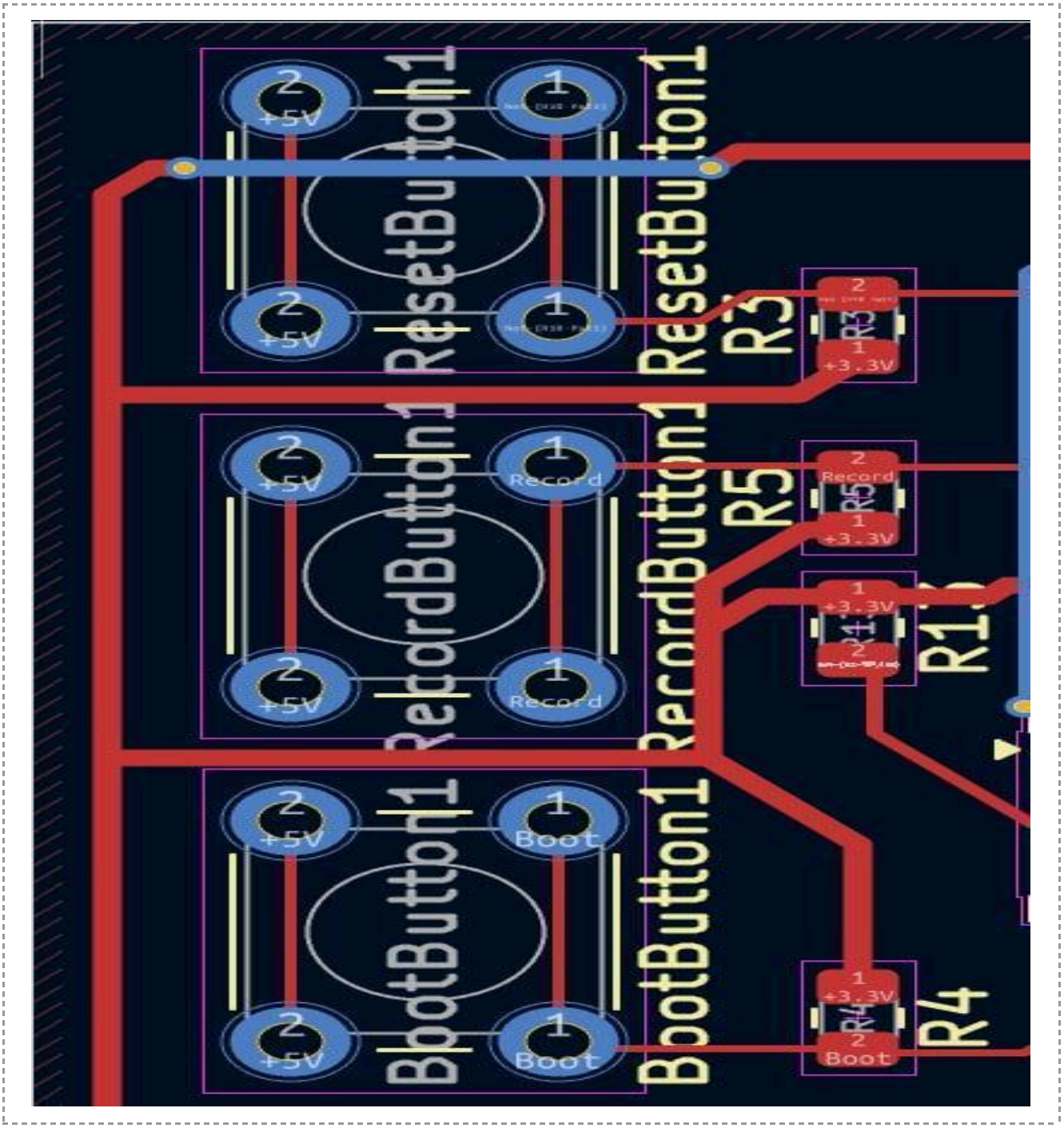


Figure 7.3

Description: Close-up of the Reset / Record / Boot button footprints and their associated pull-up resistor placement and trace routing.

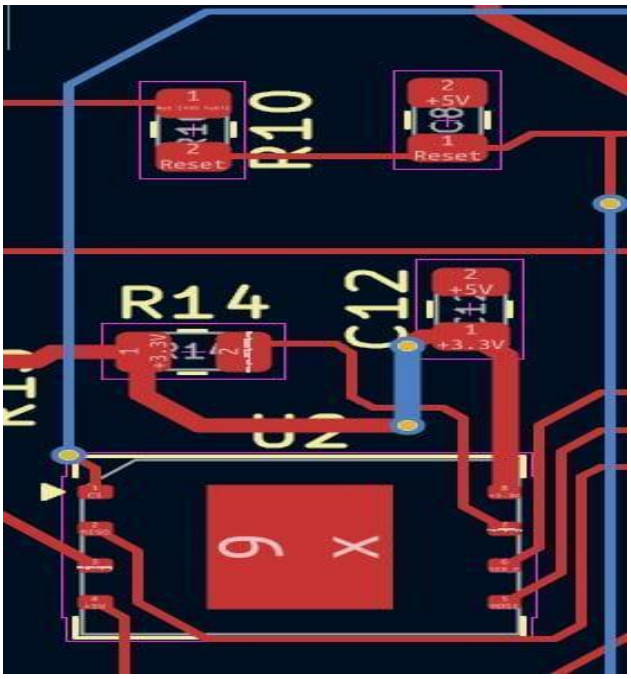


Figure 7.4

Description: Zoomed layout detail around the SPI flash (U2) and its pull-up/decoupling components (R10, R14, C12).

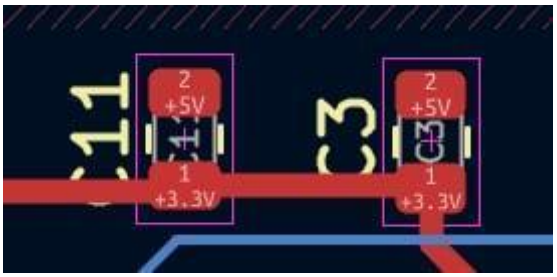


Figure 7.5

Description: Zoomed layout detail of the ESP32 power-supply decoupling capacitors (C11, C3) placed close to the module's VDD pins.

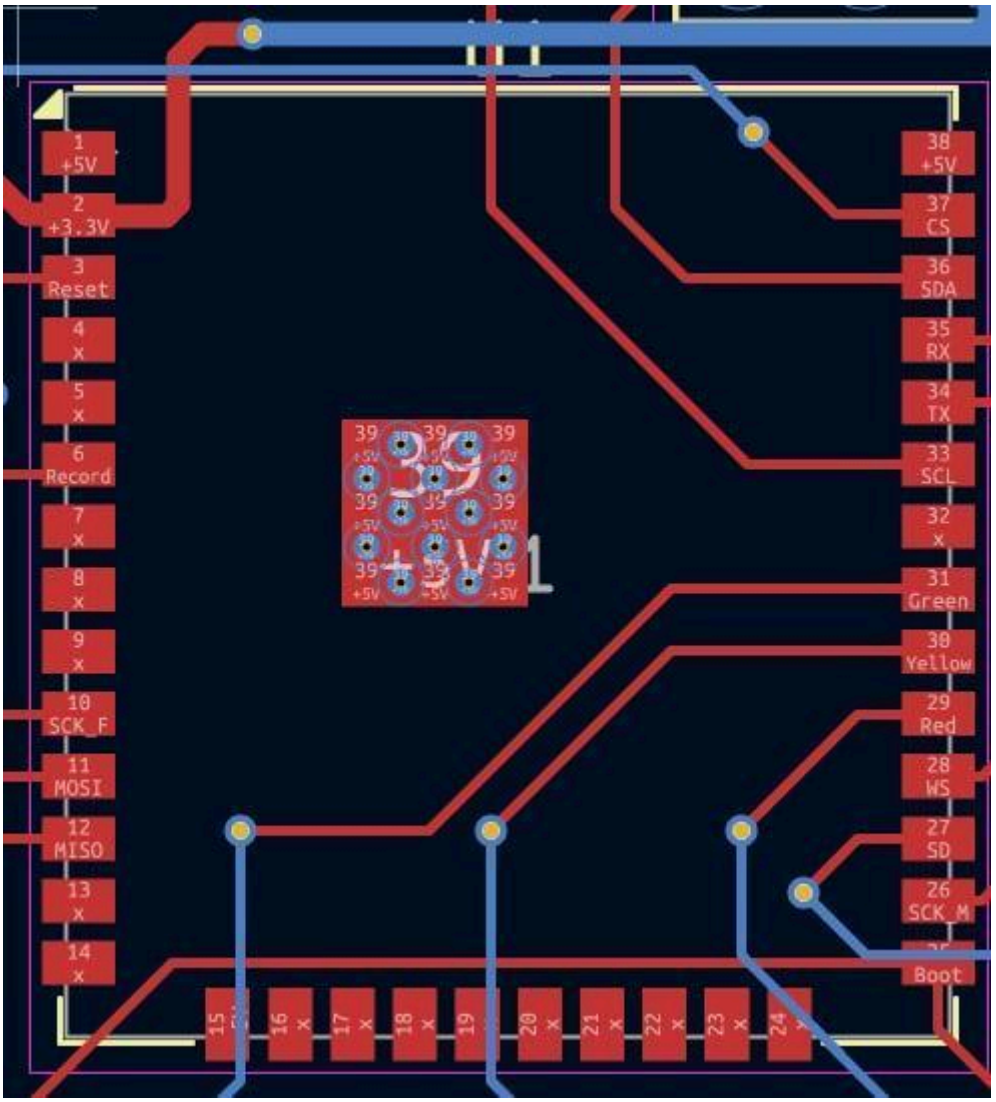


Figure 7.6

Description: Silkscreen/footprint reference for the ESP32-WROOM-32UE module (U1) with all 38 physical pin numbers and their net-name labels, matching the pin-mapping table above.

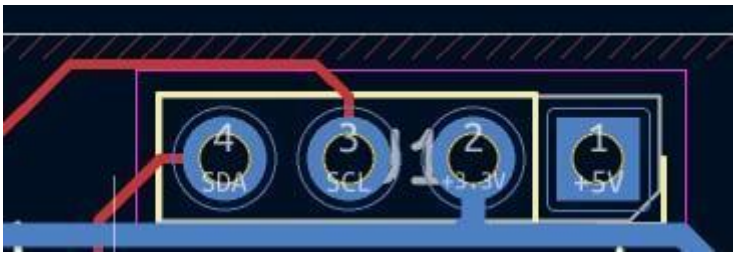


Figure 7.7

Description: Zoomed layout detail of the OLED header footprint (SDA, SCL, VCC, GND) on the PCB.

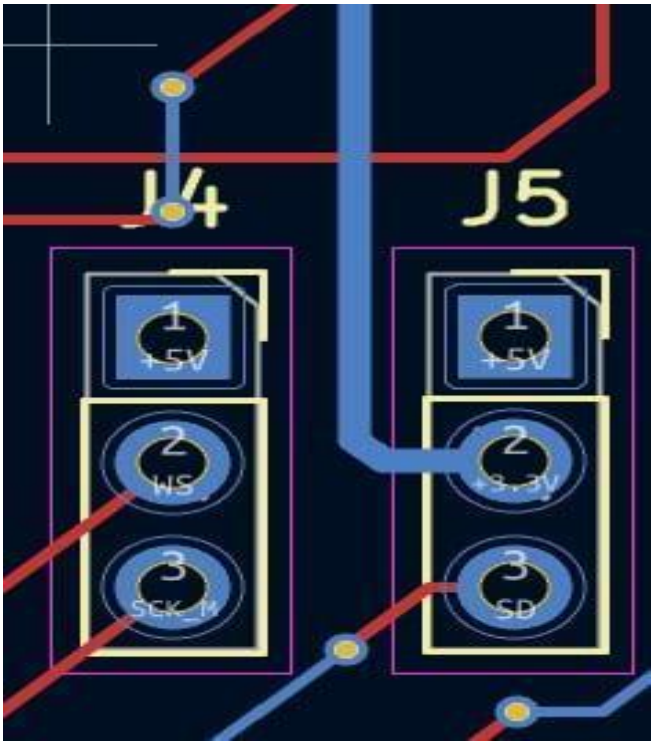


Figure 7.8

Description: Zoomed layout detail of the I2S microphone headers (J4, J5) carrying the WS, SCK, SD, and 3.3V/GND signals.

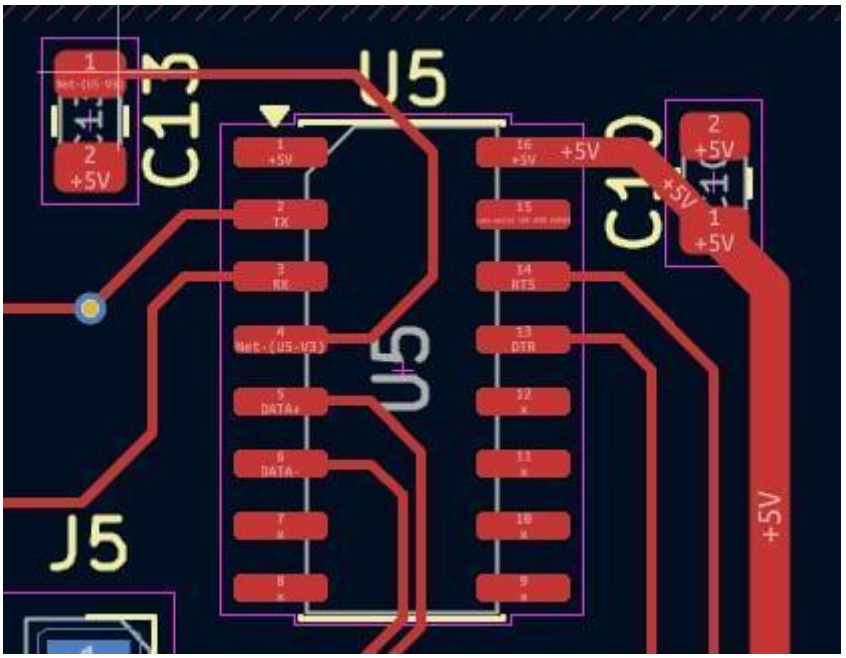


Figure 7.9

Description: Zoomed layout detail around the CH340C USB-UART IC (U5) and its decoupling capacitors (C10, C13).

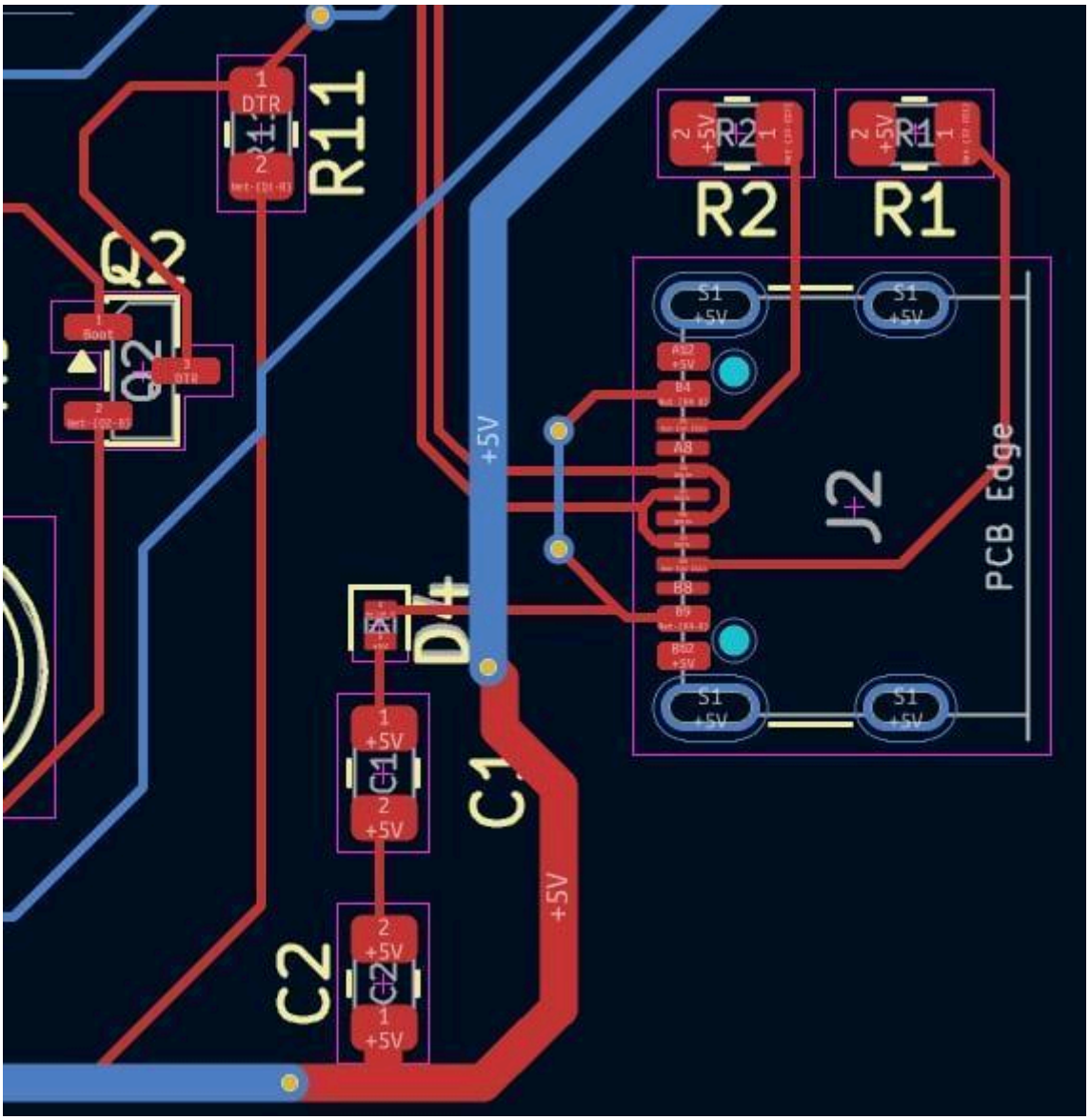


Figure 7.10

Description: Zoomed layout detail of the auto-reset transistor pair (Q1, Q2, R11) next to the USB-C connector (J2) and its ESD/filtering components.

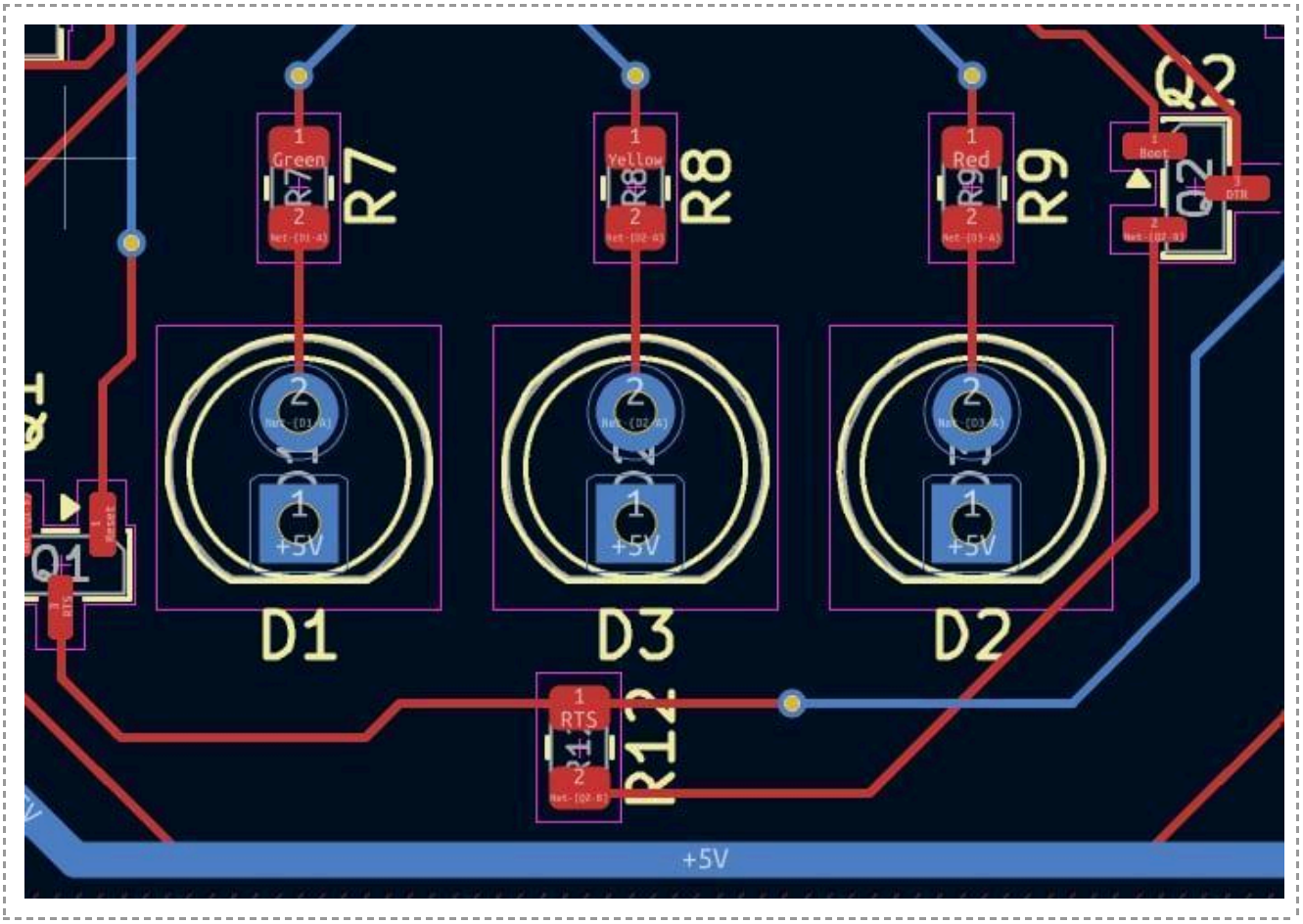


Figure 7.11

Description: Zoomed layout detail of the LED driver section (D1–D3, R7–R9, R12) showing pad and via placement around each LED.

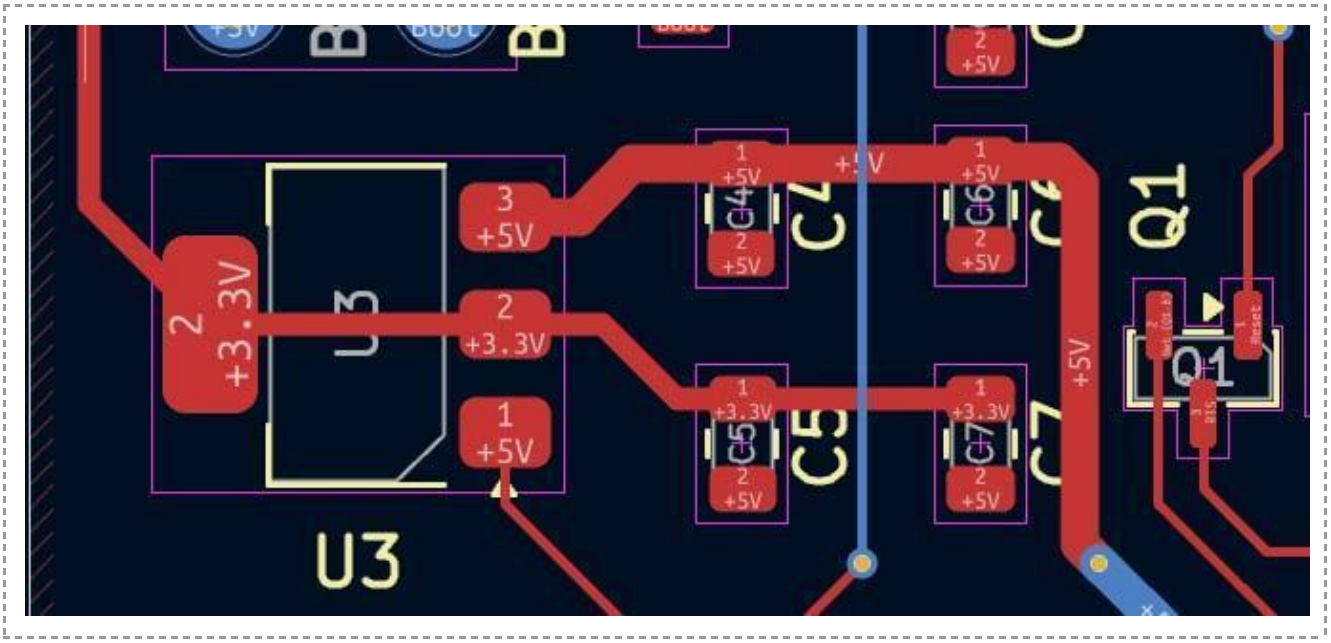
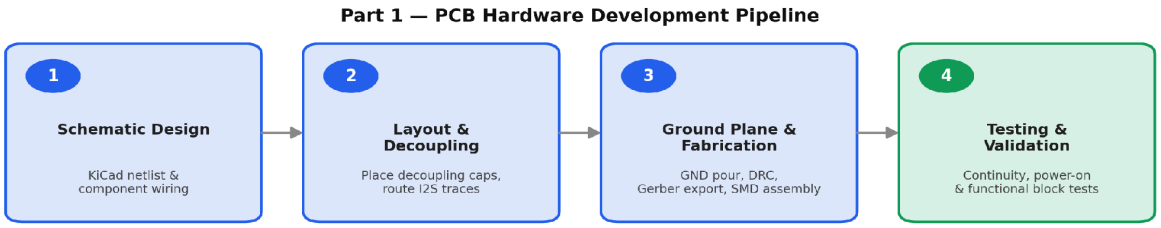


Figure 7.12

Description: Zoomed layout detail of the AMS1117 voltage regulator (U3) and its input/output decoupling capacitors (C4–C7) on the 5V-to-3.3V power section.

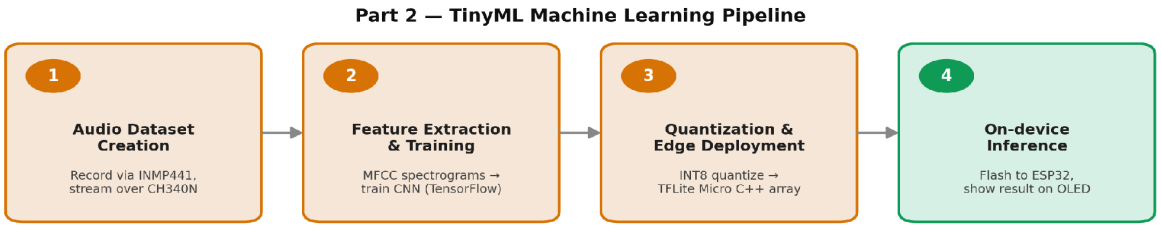
8. Dataset And Training

Part 1: PCB Hardware Development Pipeline



Module	Signal	ESP32 GPIO (physical pin)	Notes
INMP441 (I2S Mic)	SD	IO16 (pin 27)	I2S data
	WS	IO17 (pin 28)	I2S word select
	SCK_M	IO4 (pin 26)	I2S bit clock
	L/R	GND (tied low)	Selects left channel
SSD1306 (I2C OLED)	SCL	IO21 (pin 33)	I2C clock
	SDA	IO22 (pin 36)	I2C data
W25Q128JVE (SPI Flash)	CS	IO23 (pin 37)	Chip select
	SCK_F	IO25 (pin 10)	SPI clock
	MOSI	IO26 (pin 11)	
	MISO	IO27 (pin 12)	
Status LEDs	Red	IO5 (pin 29)	via R9 10k
	Yellow	IO18 (pin 30)	via R8 10k
	Green	IO19 (pin 31)	via R7 10k
Buttons	Boot	IO0 (pin 25)	R4 10k pull-up, C9 0.1µF
	Record	IO34 (pin 6)	R5 10k pull-up, input-only pin
	Reset → EN	EN pin	R3 10k pull-up, C8 0.1µF
CH340C (USB-UART)	TXD → RX net	IO1 / TXD0 (pin 35)	To PC via USB-C
	RXD ← TX net	IO3 / RXD0 (pin 34)	

Part 2: TinyML Machine Learning Pipeline



TinyML Configuration Worksheet

1. Data Acquisition

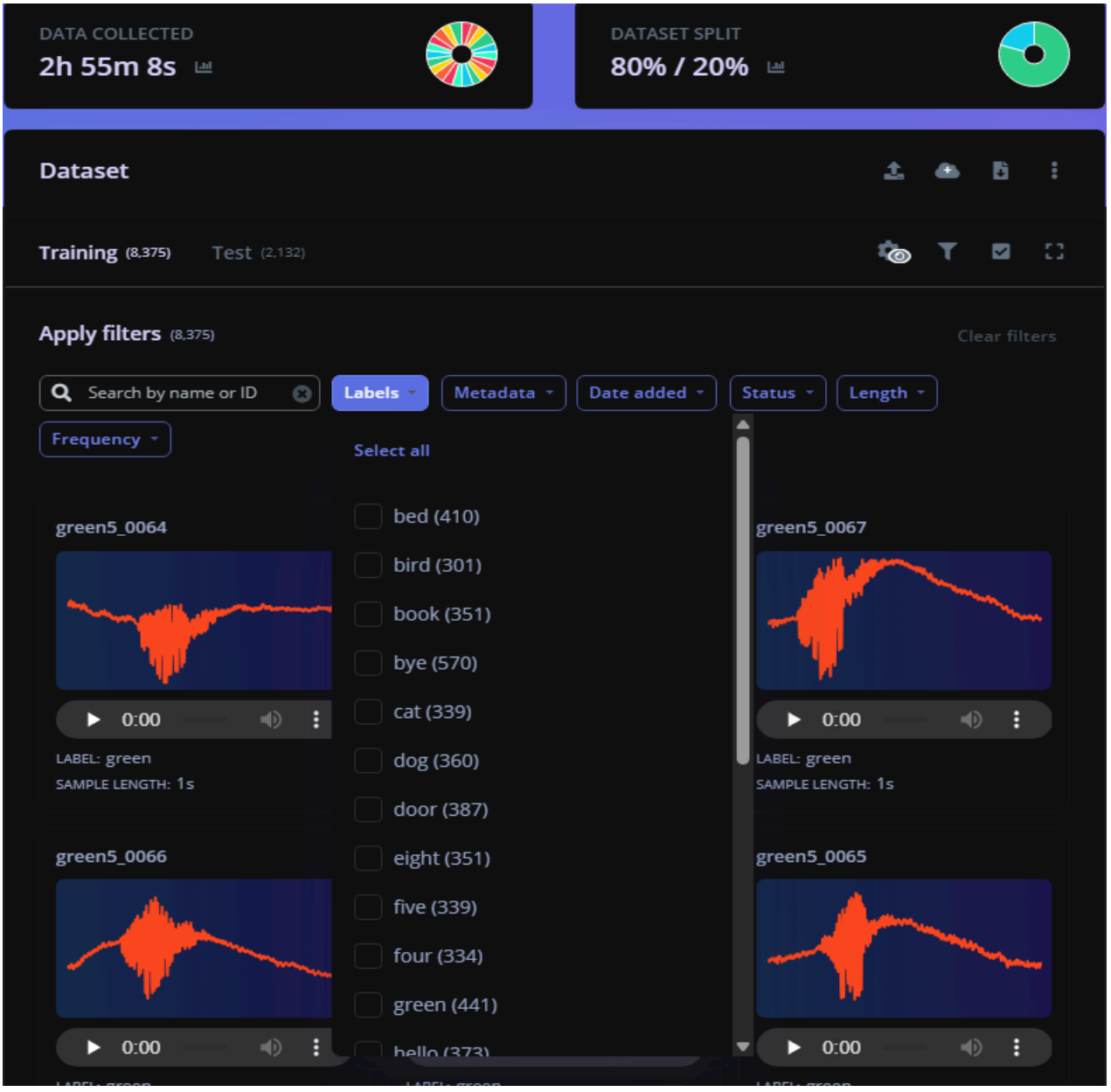


Figure 8.1

Description: ***This image illustrates the collected data, the total time taken to collect all the data labels, and the number of labels.***

Parameter	Value / Config	Notes
Sample rate	**16 kHz**	Audio captured via INMP441 using ESP32 I2S interface
Bit depth	**16-bit PCM**	I2S receives 32-bit data and converts it to 16-bit PCM
Clip length per sample	**1000 ms (1 second)**	One recording per keyword
Keyword / class labels	**23**	bed, book, bye, cat, dog, door, eight, five, four, green, hello, nine, noise, one, red, seven, six, stop, three, two, zero
Samples per class	**120–200 real + 100–150 synthetic**	Real recordings collected using INMP441 + ESP32-S3; additional synthetic samples generated using OpenAI TTS
Train / Val / Test split	**80 / 20**	Edge Impulse default dataset split

2. Feature Extraction (MFE)

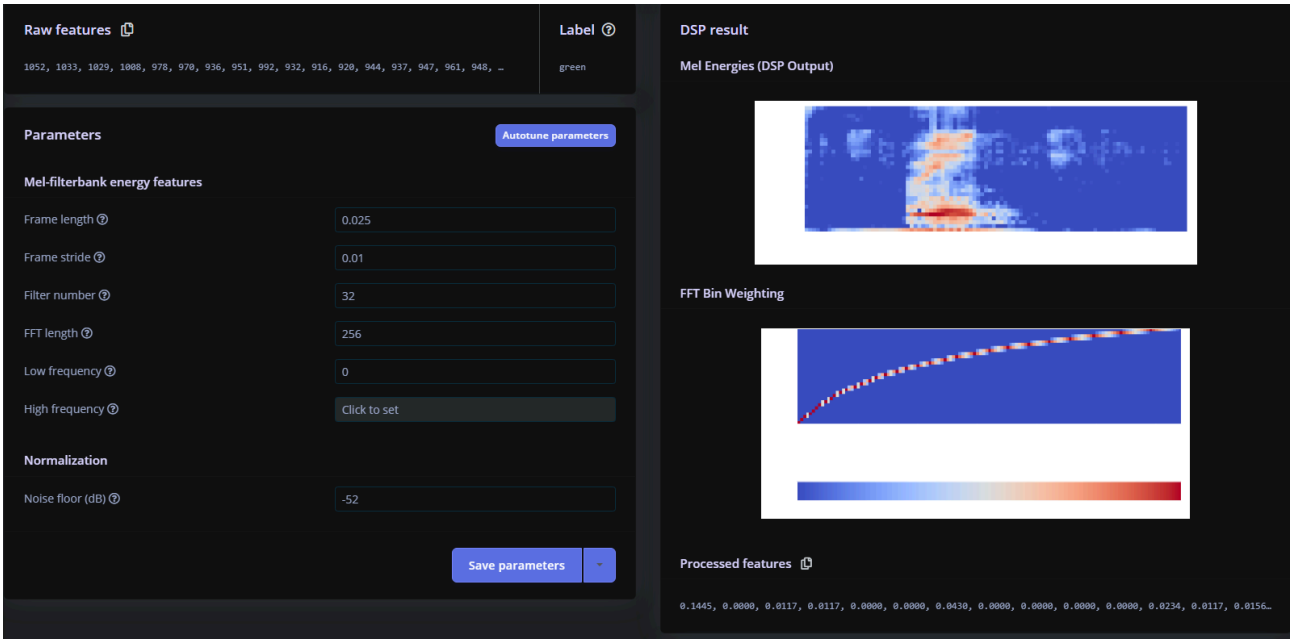


Figure 8.2

Description: ***The image illustrates the parameters of the MFE structure as well as its data type.***

Parameter	Value / Config	Notes
Frame / window length	**25 ms (0.025 s)**	Default MFE frame size
Frame stride	**10 ms (0.01 s)**	Adjacent frames overlap by 15 ms
Number of Mel filters	**32**	Number of Mel filterbank channels
FFT length	**256**	FFT size for frequency analysis
Low frequency	**0 Hz**	Lowest analyzed frequency
High frequency	**8000 Hz**	Automatically determined by Edge Impulse
Noise floor	**−52 dB**	Used for feature normalization

3. Model Training

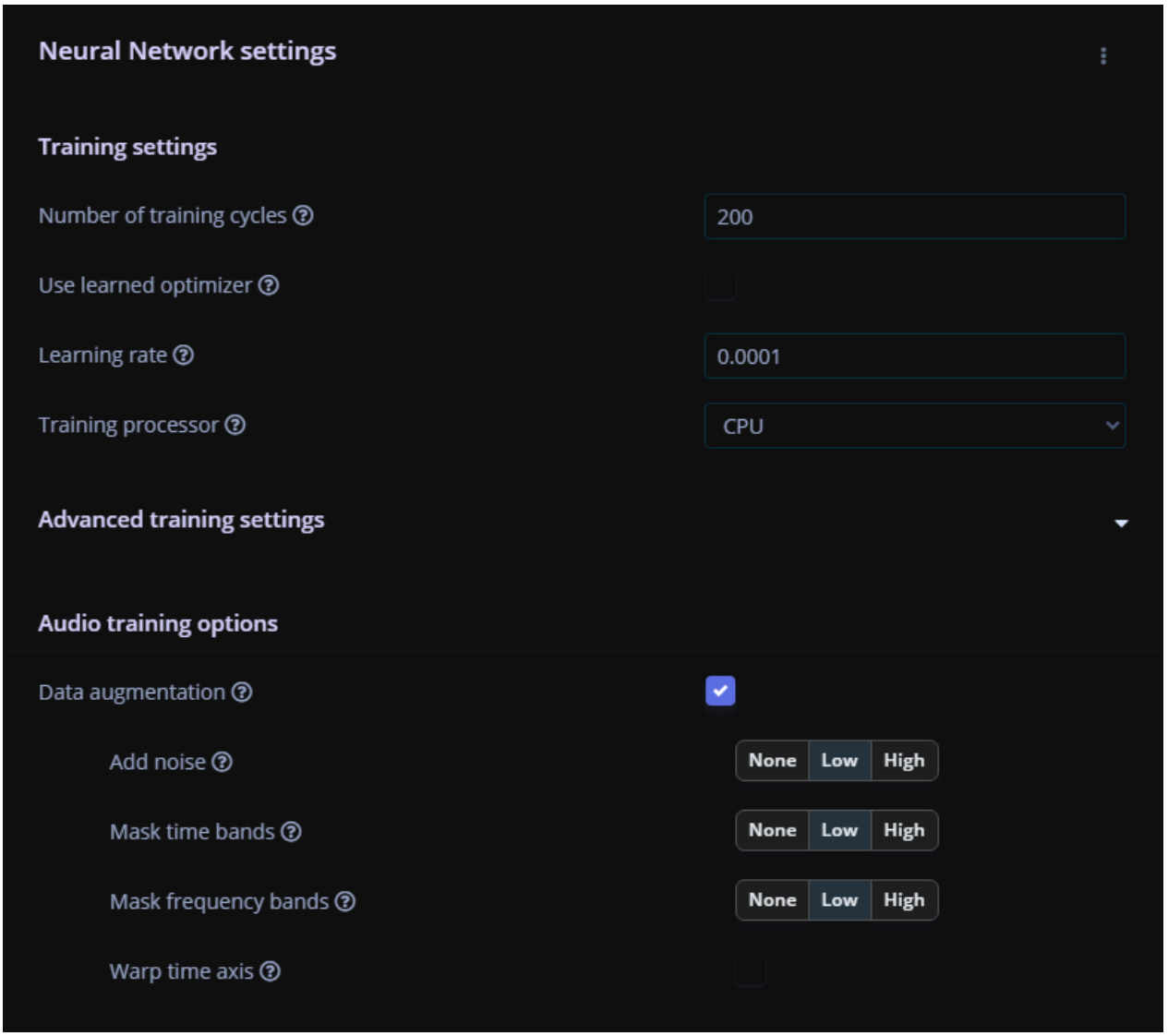
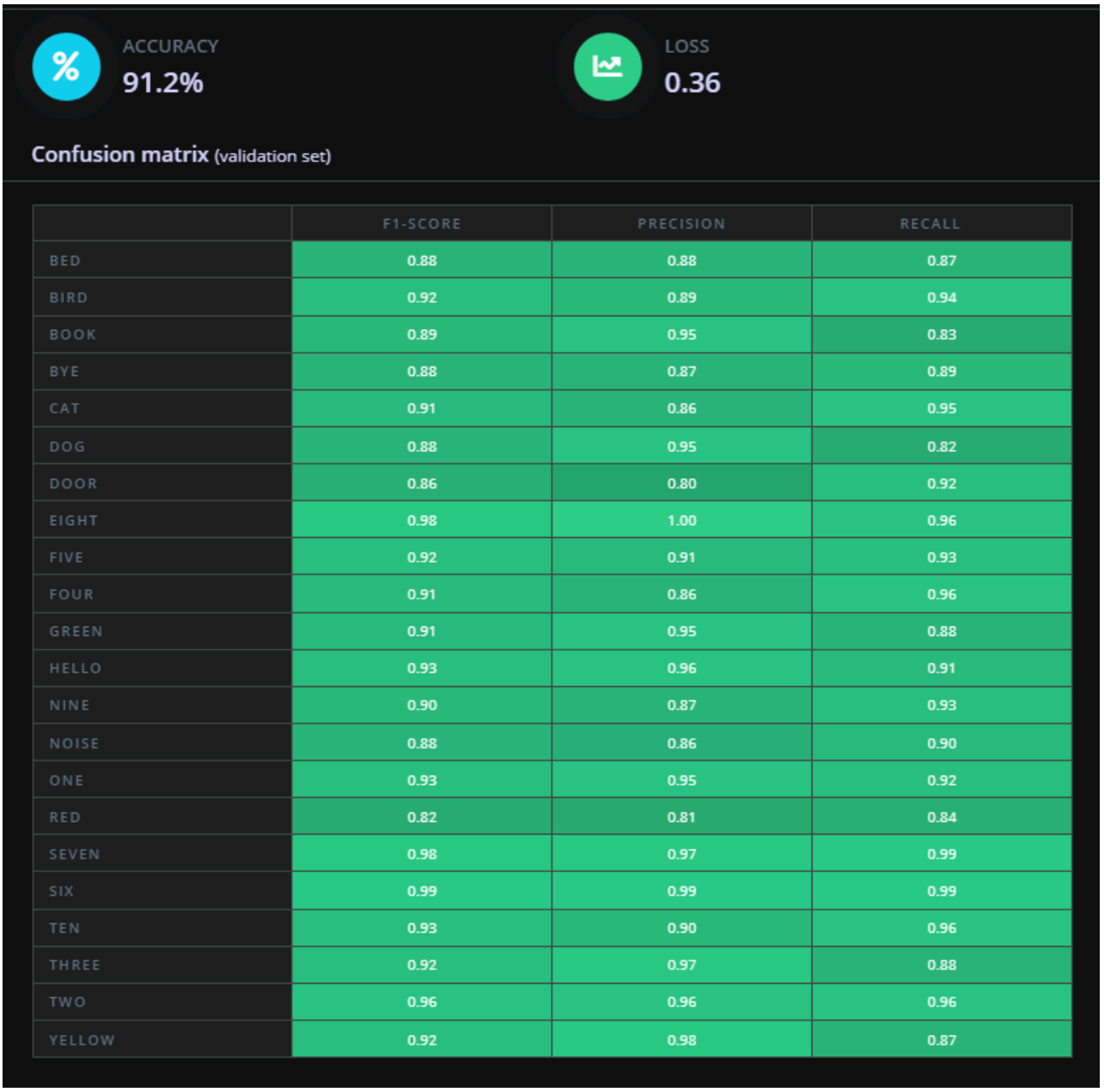


Figure 8.3

Description: ***Neural network configuration***

Parameter	Value / Config	Notes

Model architecture	**1D Convolutional Neural Network (CNN)**	Generated using Edge Impulse
Framework / version	**TensorFlow / Keras (Edge Impulse)**	Automatically generated
Epochs	**200**	Number of training cycles
Batch size	**Default (Edge Impulse)**	Default
Optimizer / learning rate	**0.0001**	Optional
Target validation accuracy	**Actual validation accuracy after training**	**91,2%**



ACCURACY
91.2%

LOSS
0.36

Confusion matrix (validation set)

	F1-SCORE	PRECISION	RECALL
BED	0.88	0.88	0.87
BIRD	0.92	0.89	0.94
BOOK	0.89	0.95	0.83
BYE	0.88	0.87	0.89
CAT	0.91	0.86	0.95
DOG	0.88	0.95	0.82
DOOR	0.86	0.80	0.92
EIGHT	0.98	1.00	0.96
FIVE	0.92	0.91	0.93
FOUR	0.91	0.86	0.96
GREEN	0.91	0.95	0.88
HELLO	0.93	0.96	0.91
NINE	0.90	0.87	0.93
NOISE	0.88	0.86	0.90
ONE	0.93	0.95	0.92
RED	0.82	0.81	0.84
SEVEN	0.98	0.97	0.99
SIX	0.99	0.99	0.99
TEN	0.93	0.90	0.96
THREE	0.92	0.97	0.88
TWO	0.96	0.96	0.96
YELLOW	0.92	0.98	0.87

Figure 8.4

Description: ***The resulting confusion matrix after training the model.***

4. Quantization & Deployment

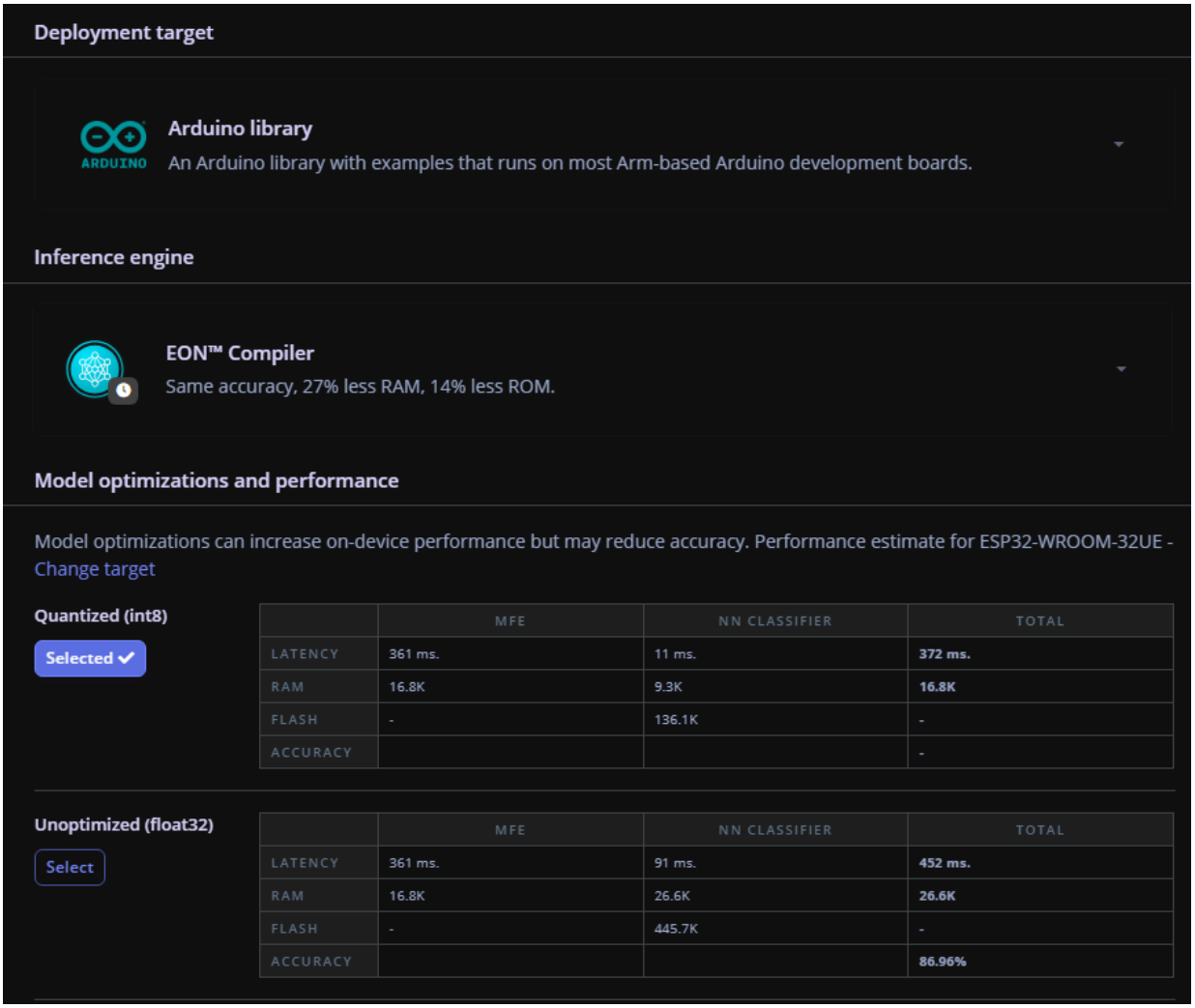


Figure 8.5

Description: ***Edge Impulse deployment configuration using EON Compiler and INT8 quantization, showing the estimated latency, RAM, and Flash memory usage on the ESP32-WROOM-32UE.***

Parameter	Value / Config	Notes
Quantization format	**INT8**	EON Compiler enabled
Exported model size	**136.1 KB**	Neural network Flash size
Inference library	**Edge Impulse Arduino Library**	Generated by Edge Impulse
Tensor arena size	**16.8 KB RAM**	Runtime RAM requirement
Confidence threshold	**Highest probability (no threshold applied)**	The label with the highest confidence is displayed
OLED output format	**Predicted label + confidence (%)**	Example: **Red – 95.4 %**

9. Firmware & Source Code

9.1 I2S Initialization

```
static bool i2s_init() {
    i2s_config_t i2s_config = {
        .mode                 = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
        .sample_rate          = EI_CLASSIFIER_FREQUENCY,
        .bits_per_sample      = I2S_BITS_PER_SAMPLE_32BIT,
        .channel_format       = I2S_CHANNEL_FMT_ONLY_LEFT,
        .communication_format = I2S_COMM_FORMAT_STAND_I2S,
        .intr_alloc_flags     = ESP_INTR_FLAG_LEVEL1,
        .dma_buf_count        = 8,
        .dma_buf_len          = 512,
        .use_apll             = true,
        .tx_desc_auto_clear   = false,
        .fixed_mclk           = 0
    };
    i2s_pin_config_t pin_config = {
        .bck_io_num   = I2S_SCK,
        .ws_io_num    = I2S_WS,
        .data_out_num = -1,
        .data_in_num  = I2S_SD,
    };
    if (i2s_driver_install(I2S_PORT, &i2s_config, 0, NULL) != ESP_OK) return false;
    if (i2s_set_pin(I2S_PORT, &pin_config) != ESP_OK) return false;
    i2s_zero_dma_buffer(I2S_PORT);
    return true;
}
```

9.2 Audio Recording

```
static void record_one_second() {

    // ← THÊM VÀO ĐÂY: xả buffer cũ 200ms
    int32_t dummy[512];
    size_t dummy_read;
    int discard = EI_CLASSIFIER_FREQUENCY * 0.2;
    int discarded = 0;
    while (discarded < discard) {
        i2s_read(I2S_PORT, dummy, sizeof(dummy), &dummy_read, portMAX_DELAY
        discarded += dummy_read / sizeof(int32_t);
    }

    const uint32_t total_samples = EI_CLASSIFIER_RAW_SAMPLE_COUNT;
    uint32_t collected = 0;
    const uint32_t chunk_samples = 512;
    int32_t raw32[chunk_samples];

    while (collected < total_samples) {
        size_t bytes_read = 0;
        i2s_read(I2S_PORT, (void*)raw32, chunk_samples * sizeof(int32_t),
                 &bytes_read, portMAX_DELAY);
        int samples = bytes_read / sizeof(int32_t);
        for (int i = 0; i < samples && collected < total_samples; i++) {
            record_buffer[collected++] = (int16_t)(raw32[i] >> 14);
        }
    }

    int64_t dc = 0;
    for (uint32_t i = 0; i < total_samples; i++) dc += record_buffer[i];
    dc /= total_samples;
    for (uint32_t i = 0; i < total_samples; i++) {
        record_buffer[i] = (int16_t)(record_buffer[i] - dc);
    }
}
```

9.3 CNN Inference And Prediction

```
signal_t signal;
signal.total_length = EI_CLASSIFIER_RAW_SAMPLE_COUNT;
signal.get_data = &get_signal_data;

ei_impulse_result_t result = { 0 };
EI_IMPULSE_ERROR r = run_classifier(&signal, &result, debug_nn);
if (r != EI_IMPULSE_OK) {
    Serial.printf("Loi classifier: %d\n", r);
    oled_show("Loi!", "Thu lai");
    led_set(true, false, false); // lỗi thì coi như quay lại sẵn sàng
    while (digitalRead(BUTTON_PIN) == LOW) delay(10);
    return;
}
```

```
float max_val = 0;
int max_idx = 0;
for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    if (result.classification[i].value > max_val) {
        max_val = result.classification[i].value;
        max_idx = i;
    }
}
const char* label = result.classification[max_idx].label;
```

9.4 Oled Display

```
char percent_str[16];
snprintf(percent_str, sizeof(percent_str), "%.1f%%", max_val * 100);
oled_show(label, percent_str);
```

9.5 Main Program

```
led_set(false, false, true);

// Nếu label trùng tên màu, đổi sang đèn tương ứng (tắt xanh trước đó)
if (strcmp(label, "red") == 0) {
    led_set(true, false, false);
} else if (strcmp(label, "yellow") == 0) {
    led_set(false, true, false);
} else if (strcmp(label, "green") == 0) {
    led_set(false, false, true); // giữ nguyên xanh
}

while (digitalRead(BUTTON_PIN) == LOW) delay(10); // đợi nhả nút

delay(1500); // giữ kết quả 1.5s cho dễ đọc
oled_show("San sang", "Nhan nut de noi");
led_set(true, false, false);   // quay lại sẵn sàng: đèn đỏ
```

10. Project Results

This section presents the final implementation of the TinyML-based speech recognition system running on the ESP32-WROOM-32UE. The system was tested under three operating states to verify the complete speech recognition workflow, including standby, audio processing, and recognition result display.

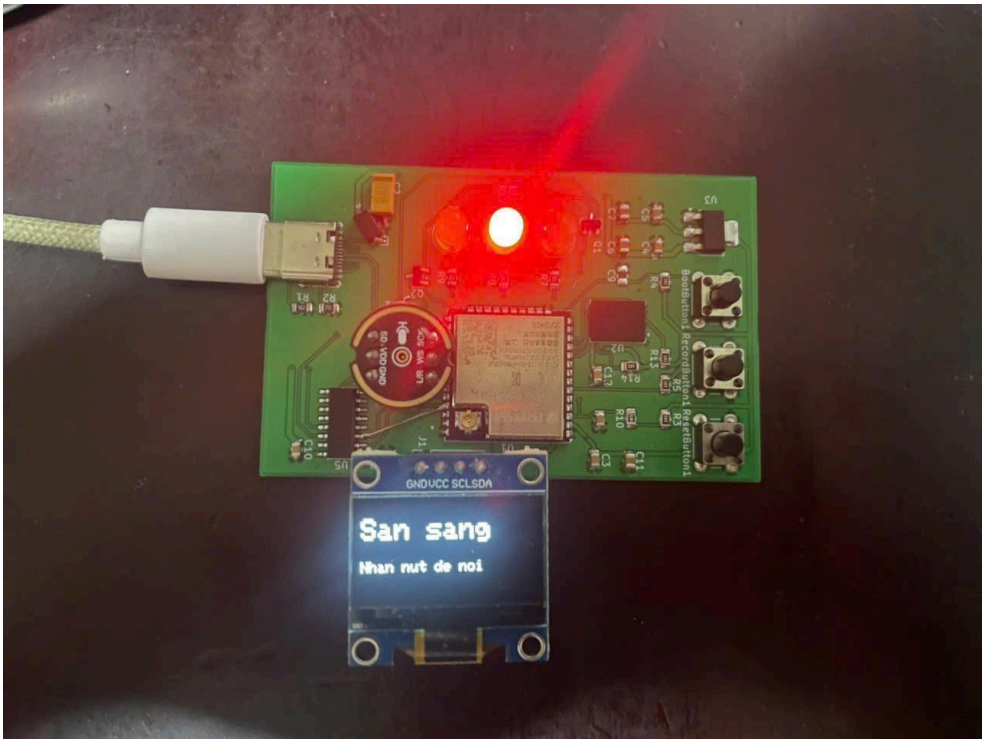


Figure 10.1. Standby State

The system is initialized and ready to receive a voice command. The OLED displays the message "Ready – Press button to speak", while the status LED indicates that the device is waiting for user input.

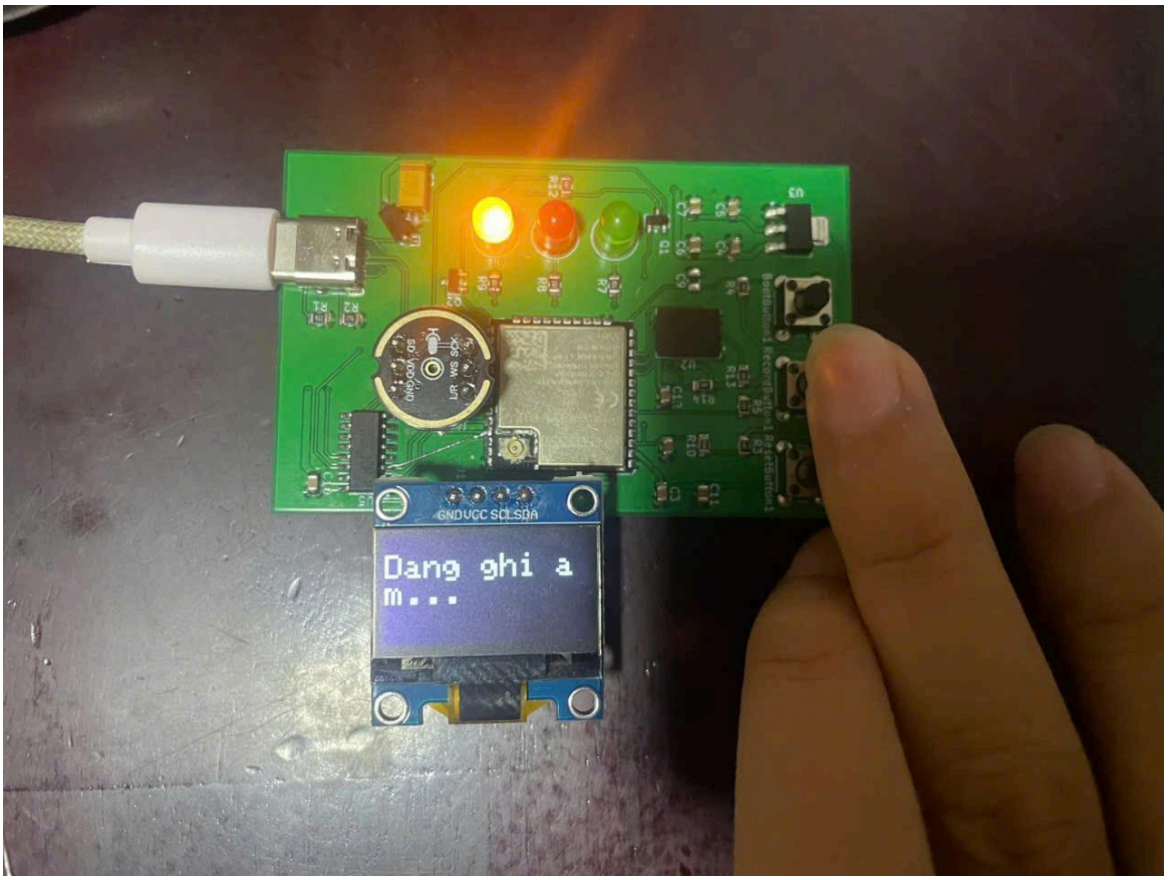


Figure 10.2. Audio Recording and Processing State

After the user presses the button, the ESP32 records a one-second audio sample from the INMP441 microphone. The captured signal is processed using Mel Filterbank Energy (MFE) feature extraction, and the trained CNN model performs on-device inference. During this stage, the OLED displays "Recording..." or "Recognizing...", indicating that the system is processing the speech input.

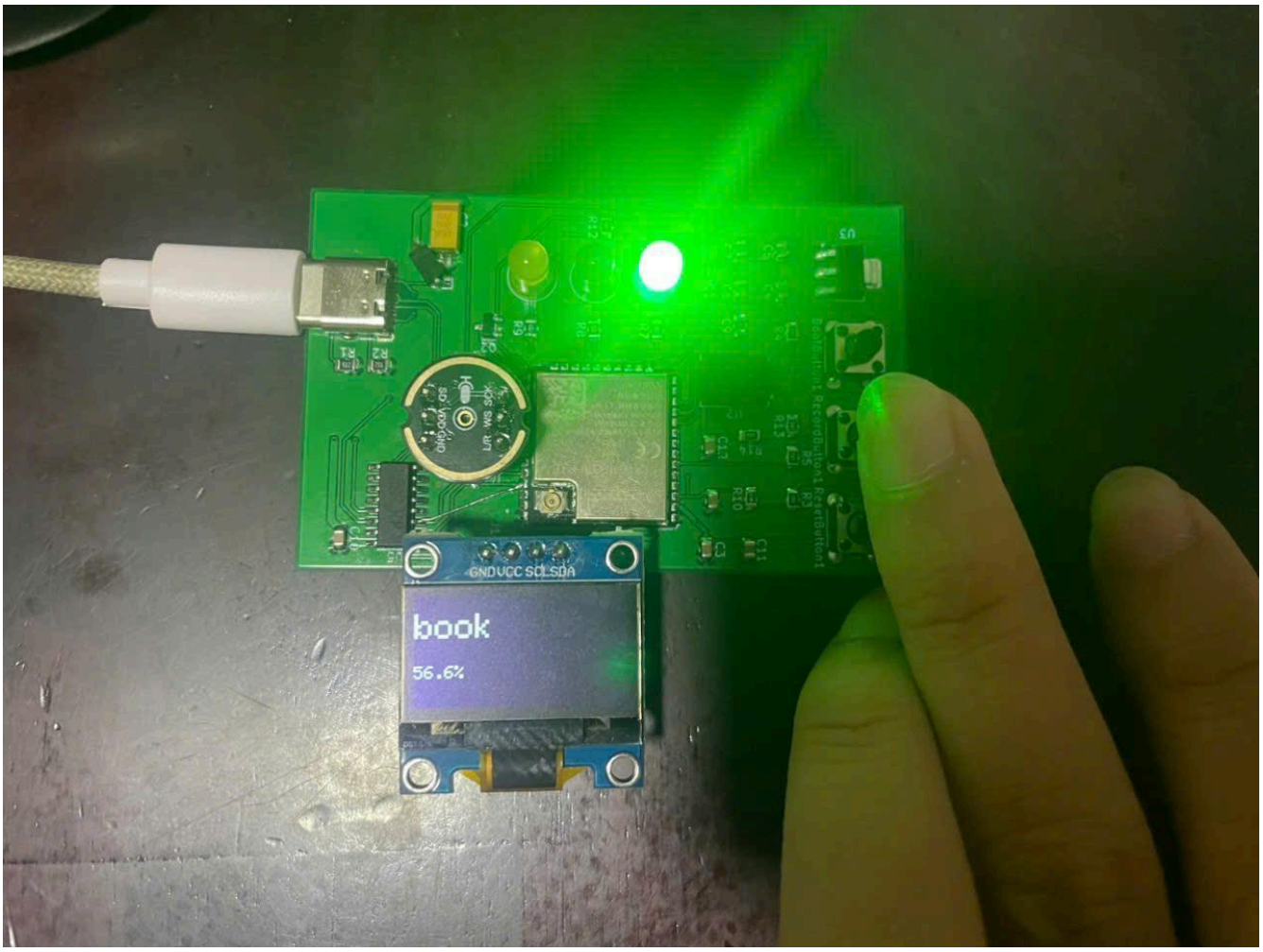


Figure 10.3. Recognition Result

After inference is completed, the OLED displays the predicted keyword together with its confidence score. The corresponding LED is activated according to the recognized command, providing additional visual feedback. This demonstrates that the proposed TinyML system can perform real-time offline speech recognition directly on the ESP32 without requiring cloud connectivity.

Experimental Summary

The experimental results confirm that the proposed system successfully integrates audio acquisition, MFE feature extraction, CNN inference, and user feedback into a complete embedded speech recognition application. The implementation achieves real-time keyword recognition while operating entirely on the ESP32 microcontroller, demonstrating the practicality of TinyML for low-power embedded devices.

11. Conclusion

This project successfully developed a TinyML-based speech command recognition system using the ESP32-WROOM-32UE microcontroller and the INMP441 digital microphone. The proposed system integrates audio acquisition, Mel Filterbank Energy (MFE) feature extraction, and a lightweight 1D Convolutional Neural Network (CNN) trained using the Edge Impulse platform. The trained model was deployed on the ESP32, enabling real-time on-device inference without requiring cloud processing.

Experimental results demonstrate that the system can recognize 22 predefined voice commands with satisfactory accuracy while maintaining low memory usage and fast inference time, making it suitable for resource-constrained embedded applications. The recognition results are displayed on a 0.96-inch OLED screen together with the confidence score, while LEDs provide additional visual feedback according to the predicted command.

This project demonstrates the feasibility of combining TinyML and embedded systems to implement efficient offline speech recognition. Compared with cloud-based solutions, the proposed approach offers lower latency, improved privacy, and reduced communication requirements.